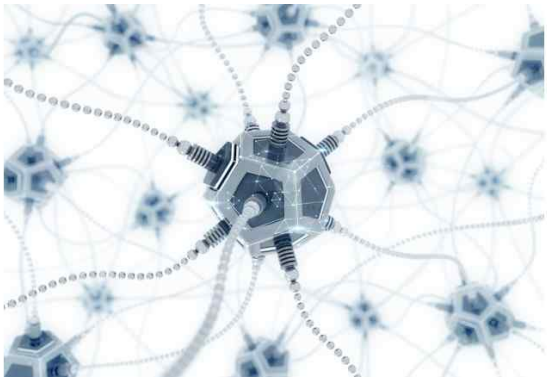
Chapter 03
Deep Learning 알고리즘의 이해

여기서는 딥러닝의 기본 알고리즘을 딥러닝 7 공식을 통해 이해해 보고, 파이토치의 활용 방법을 소개합니다.

01 딥러닝 개요

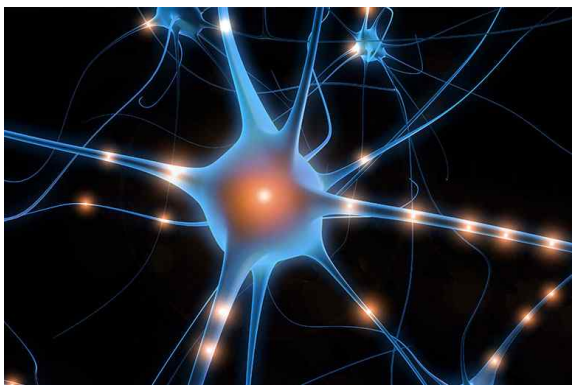


▶ 출처 : https://www.discovermagazine.com/technology/we-almost-gave-up-on-building-artificial-brains

인공 신경망은 딥러닝의 약진에 의해 최근 몇 년 동안 주목을 받아왔습니다. 그러면 인공 신경망은 무엇이고 어떻게 만들어졌을까요? 여기서는 인공 신경망의 바탕이 되는 실제 생체 신경의 구조와 구성 요소를 살펴보고 그것들이 어떻게 인공 신경의 구조와 구성요소에 대응이 되는지 살펴봅니다.

01 인공 신경망이란?

독자 여러분은 지금까지 왜 사람에게는 아주 간단하지만 컴퓨터에게는 상상할 수 없을 정도로 어려운 일들이 있는지 궁금해 한 적이 있나요? 인공 신경망(ANN's : Artificial neural networks)은 인간의 중앙 신경계로부터 영감을 얻어 만들어졌습니다. 생체 신경망과 같이 인공 신경망은 커다란 망으로 함께 연결되어 있는 인공 신경을 기반으로 구성됩니다. 개개의 인공 신경은 생체 신경과 같이 간단한 신호 처리를 할 수 있도록 구현되어 있습니다.



https://www.allerin.com/blog/4-benefits-of-using-artificial-neural-nets

인공 신경망으로 할 수 있는 일들

그러면 우리는 인공 신경망으로 무엇을 할 수 있을까요? 인공 신경망은 많은 문제 영역에 성공적으로 적용되어 왔습니다. 예를 들어 다음과 같은 문제들에 적용되었습니다.

- 패턴 인식에 의한 데이터 분류
  그림에 있는 이것은 나무인가?

- 입력 데이터가 일반적인 패턴과 맞지 않았을 때의 이상 감지
  트럭 운전사가 잠들 위험이 있는가?
  이 지진은 일반적인 땅의 움직임인가 아니면 커다란 지진인가?

- 신호 처리
  신호 거르기
  신호 분리하기
  신호 압축하기

- 예측과 예보에 유용한 목표 함수 접근
  이 폭풍은 태풍으로 변할 것인가?

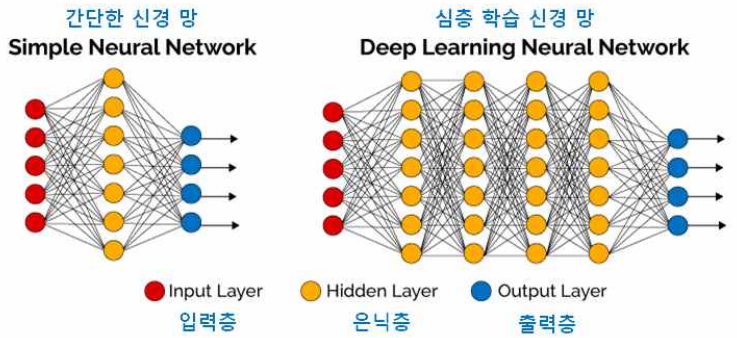
이런 문제들은 조금은 추상적으로 들립니다. 그래서 몇 가지 실제로 적용된 응용 예들을 보도록 합니다. 인공 신경망은 다음과 같은 것들을 할 수 있습니다.

- 얼굴 확인하기
- 음성 인식하기
- 손 글씨 읽기
- 문장 번역하기
- 게임 하기(보드 게임이나 카드 게임)
- 자동차나 로봇 제어하기
- 그리고 더 많은 것들!

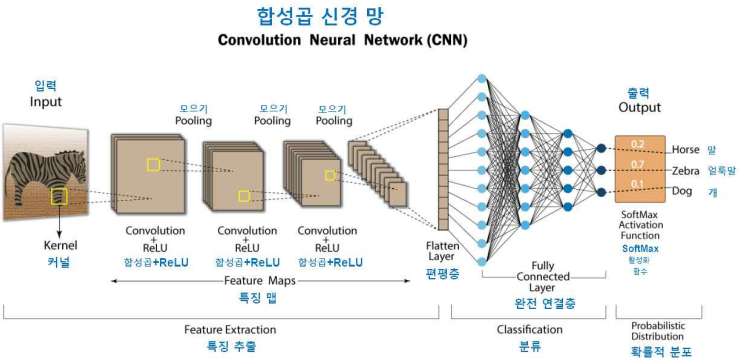
인공 신경망을 이용하면 세상에 있는 많은 문제들을 해결할 수 있습니다. 독자 여러분도 해결하고 싶은 문제가 있다면, 인공 신경망을 이용해 해결할 가능성이 있습니다. 인공 신경망을 통한 문제 해결은 이제 선택이 아닌 필수가 되어가고 있으며, 인공 신경망을 통한 문제 해결 능력은 여러분에게 더 많은 기회를 줄 것입니다.

인공 신경망의 구조

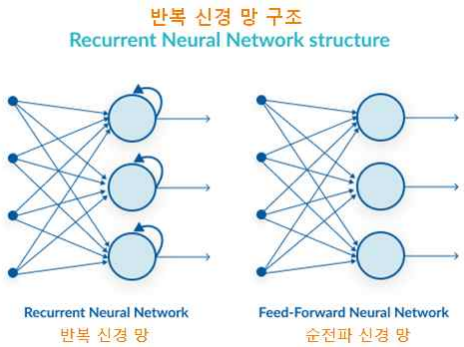
인공 신경망을 구성하는 방법은 다양합니다. 예를 들어 다음과 같은 형태로 인공 신경망을 구성할 수 있습니다. 다음 그림에서 노란색 노드로 표현된 은닉 층이 2층 이상일 때 심층 신경망(DNN)이라고 합니다.



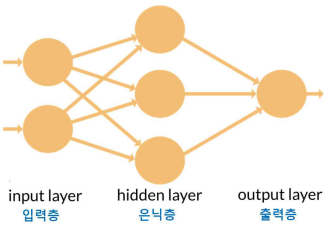
다음은 CNN 형태의 인공 신경망입니다. CNN은 이미지 인식에 뛰어난 인공 신경망으로 이미지의 특징을 뽑아내는 인공 신경망과 분류를 위한 인공 신경망으로 구성됩니다.



다음은 RNN 형태의 인공 신경망입니다. 아래 그림에서 왼쪽에 있는 그림은 RNN 형태의 신경망으로 노드에서 나온 값이 다시 되먹임 되는 형태로 인공 신경망이 구성됩니다. 오른쪽에 있는 그림은 한 방향으로만 신호가 흐르는 기본적인 인공 신경망입니다. RNN 형태의 인공 신경망은 문장 인식에 뛰어난 인공 신경망입니다.



인공 신경망은 구성 방법에 따라 동작 방식도 달라집니다. 가장 간단한 인공 신경망의 구조는 신호가 한 방향으로 흐르는 인공 신경망으로 다음과 같은 형태입니다.



일반적으로 인공 신경망은 3개의 층으로 구성됩니다. 각각 입력 층, 은닉 층, 출력 층이라고 합니다. 입력 층은 입력 신호를 받아서 다음 층에 있는 은닉 층으로 보냅니다. 은닉 층은 하나 이상 존재할 수 있습니다. 마지막에는 결과를 전달하는 출력 층이 옵니다.

02 인공 신경망의 학습 방법

전통적인 알고리즘들과는 달리 인공 신경망은 프로그래머의 의도대로 작업하도록 '프로그램 되거나' 또는 '구성되거나' 할 수 없습니다. 인간의 뇌처럼 인공 신경망은 하나의 일을 수행할 방법을 직접 배워야 합니다. 일반적으로 인공 신경망의 학습 방법에는 3가지 전략이 있습니다.

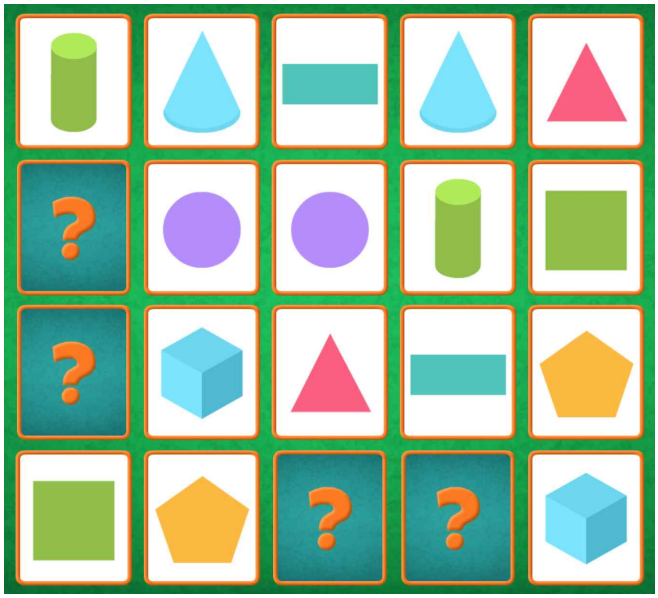
지도 학습

가장 간단한 학습 방법입니다. 미리 알려진 결과들이 있는 충분히 많은 데이터가 있을 때 사용하는 방법입니다. 지도 학습은 다음 순서로 진행됩니다. ❶ 하나의 입력 데이터를 처리합니다. ❷ 출력값을 미리 알려진 결과와 비교합니다. ❸ 인공 신경망을 수정합니다. ❹ 이 과정을 반복합니다. 이것이 지도 학습 방법입니다. 예를 들어 엄마가 어린 아이에게 그림판을 이용하여 사물을 학습시키는 방법은 지도 학습과 같습니다. 한글, 숫자 등에 대한 학습도 지도 학습의 형태입니다. 아래에 있는 그림판에는 동물, 과일, 채소 그림이 있고 해당 그림에 대한 이름이 있습니다. 아이에게 고양이를 가리키면서 '고양이'라고 알려주는 과정에서 아이는 학습을 하게 됩니다. 이와 같은 방식으로 인공 신경망도 학습을 시킬 수 있으며, 이런 방법을 지도 학습이라고 합니다.



비지도 학습

비지도 학습은 입력 값이 목표 값과 같을 때 사용하는 학습 방법입니다. 예를 들어, 메모리 카드 게임을 하는 방식을 생각해 봅니다. 메모리 카드 게임을 할 때 우리는 그림에 표현된 사물의 이름을 모르는 상태로 사물의 형태를 통째로 기억해야 합니다. 그리고 같은 그림을 찾아 내며 게임을 진행하게 됩니다. 이와 같이 입력 값과 출력 값이 같은 형태의 데이터를 학습할 때, 즉, 입력 값을 그대로 기억해 내야 하는 형태의 학습 방법을 비지도 학습이라고 합니다.



https://www.curiousworld.com/blog/importance-playing-memory-games

강화 학습

인공 신경망이 익숙하지 않은 환경에서 시행착오를 통해 이익이 되는 동작을 취할 확률은 높이고 손해가 되는 동작을 취할 확률은 낮추게 하는 학습 방법입니다. 즉, 이익이 되는 동작을 강화해가는 학습 방법입니다. 예를 들어, 우리가 익숙하지 않은 환경에서 어떤 동작을 취해야 하는지 모를 때, 일단 할 수 있는 동작을 취해보고 그 동작이 유리한지 불리한지를 체득하는 형태의 학습 방식과 같습니다. 이 과정에서 유리한 동작은 기억해서 점점 더 하게 되고 불리한 동작도 기억해서 점점 덜 하게 됩니다.





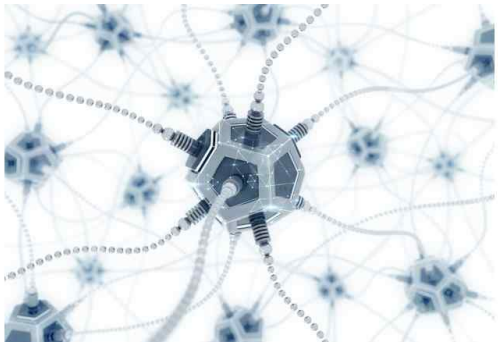
https://www.unict.it/it/didattica/news/lauree-magistrali-giornate-di-orientamento
https://safedownautobelay.com/benefits-of-getting-children-climbing/

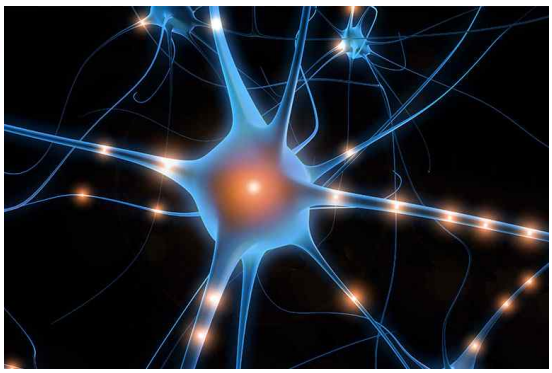
03 인공 신경 살펴보기

앞에서 우리는 인공 신경망에 대해 살펴보았습니다. 그러면 인공 신경망은 무엇으로 구성될까요? 여기서는 인공 신경망을 구성하는 인공 신경에 대해 생물학적 신경과 비교해 보면서 그 내부 구조를 살펴보도록 합니다.

인공 신경과 생물학적 신경

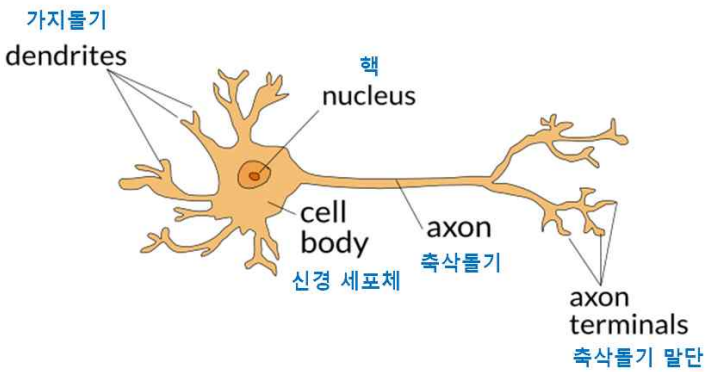
인공 신경망의 구성요소는 인공 신경입니다. 인공 신경이라는 이름은 생물학적 신경으로부터 얻어졌습니다. 인공 신경은 우리 두뇌의 구성 요소 중 하나인 생물학적 신경의 동작을 따라 만들어진 모형(model)입니다. 즉, 인공 신경은 생물학적 신경의 모형입니다.



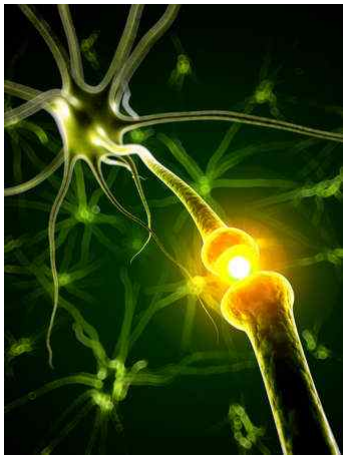


https://www.discovermagazine.com/technology/we-almost-gave-up-on-building-artificial-brains
https://www.allerin.com/blog/4-benefits-of-using-artificial-neural-nets

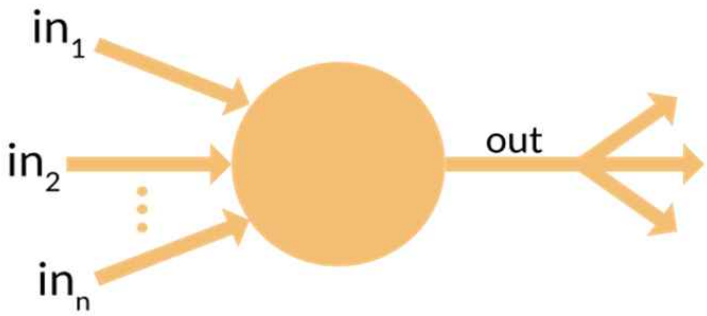
생물학적 신경은 신호를 받기 위한 여러 개의 가지 돌기(dendrities), 입력받은 신호를 처리하기 위한 신경 세포체(cell body), 다른 신경들로 신호를 내보내기 위한 축삭돌기(axon)와 축삭돌기 말단으로 구성됩니다.



특히 축삭돌기 말단과 다음 신경의 가지 돌기 사이의 틈을 시냅스라고 합니다.

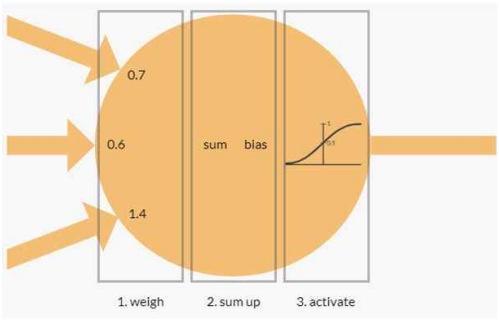


시냅스는 신경결합 부라고도 하며 한 신경에서 다른 신경으로 신호를 전달하는 연결지점을 말합니다. 인공 신경은 데이터를 받기 위한 여러 개의 입력 부, 입력받은 데이터를 처리하는 처리부, 그리고 여러 개의 다른 인공 신경들로 연결될 수 있는 하나의 출력부를 가집니다. 특히 인공 신경의 출력부에는 다음 인공 신경의 입력부에 맞는 형태의 데이터 변환을 위한 활성화함수가 있습니다.



인공 신경 내부 살펴보기

이제 인공 신경 안으로 들어가 봅니다. 어떻게 인공 신경은 입력을 처리할까요? 독자 여러분은 하나의 인공 신경 안에서 그 계산들이 실제로 얼마나 간단한지 알면 깜짝 놀랄 수도 있습니다. 인공 신경은 세 개의 처리 단계를 수행합니다.



❶ 각각의 입력 값은 가중치에 의해 커지거나 작아집니다.

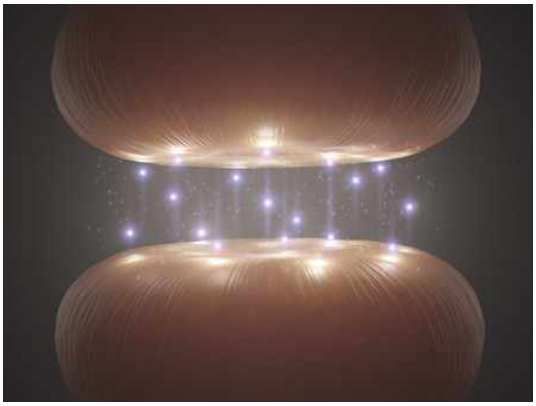
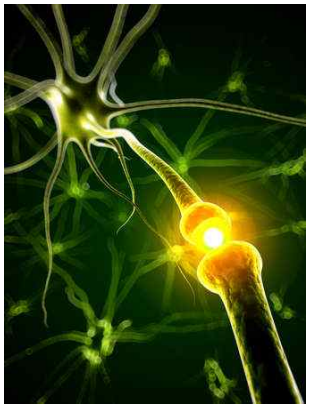
하나의 입력 신호(데이터)가 들어올 때 그 신호는 그 입력에 할당된 하나의 가중치(weight)에 의해 곱해집니다. 예를 들어, 하나의 인공 신경이 그림과 같이 3 개의 입력을 가진다면, 그 인공 신경은 각 입력에 적용될 수 있는 3개의 가중치를 가집니다. 학습 과정에서 인공 신경망은 결과 값과 목표 값의 오차를 기반으로 가중치들의 크기를 조정합니다. 생물학적 신경의 가지 돌기가 그 두께에 따라 신호가 더 잘 전달되거나 덜 전달되는 것처럼 인공 신경의 가중치도 그 값에 따라 신호(데이터)가 커지거나 작아집니다. 가중치는 다른 말로 강도(strength)라고도 합니다. 즉, 가중치는 입력 신호가 전달되는 강도를 결정합니다. 입력 신호가 작더라도 가중치가 크면 신호가 커지며, 입력 신호가 크더라고 가중치가 작으면 내부로 전달되는 신호는 작아집니다. 인공 신경의 가중치는 생물학적 신경의 가지 돌기의 두께로 비유할 수 있습니다. 인공 신경의 학습과정은 입력값이 출력값에 맞도록 반복적으로 가중치 값의 크기를 조절하는 과정입니다.

❷ 모든 입력 신호들은 더해집니다.

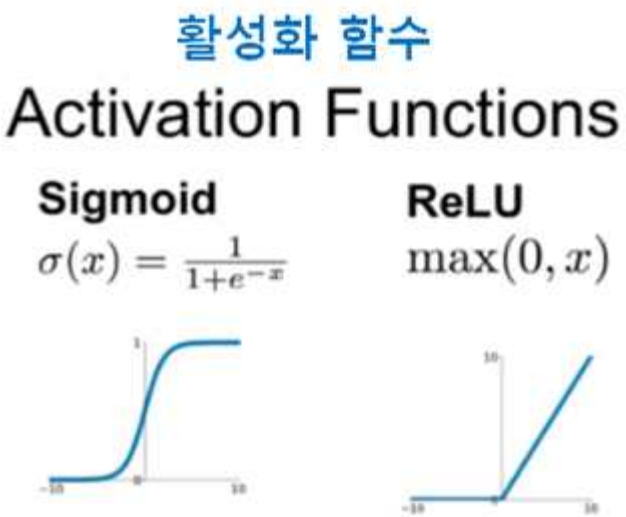
가중치에 의해 곱해진 입력 신호들은 하나의 값으로 더해집니다. 그리고 추가적으로 보정 값(offset)도 하나 더해집니다. 이 보정 값은 편향(bias)이라고 불립니다. 인공 신경망은 학습 과정에서 편향도 조정합니다. 편향은 하나로 더해진 입력 신호에 더해지는 신호로 신호를 좀 더 크게 하거나 또는 좀 더 작게 하는 역할을 합니다. 즉, 신호를 조금 더 강화하거나 조금 더 약화하는 역할을 합니다. 편향은 가중치의 한 형태입니다.

❸ 신호를 활성화합니다.

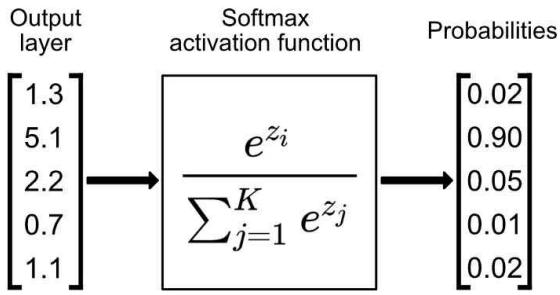
앞에서 더해진 입력신호들은 활성화함수를 거쳐 하나의 출력 신호로 바뀝니다. 활성화 함수는 신호 전달 함수라고도 하며 신호의 형태를 다른 인공 신경의 입력에 맞게 변경하여 출력하는 역할을 합니다. 생물학적 신경을 시냅스가 연결하는 것처럼 활성화함수는 인공 신경을 연결하는 역할을 수행합니다.

다음은 인공 신경망에 사용되는 활성화함수입니다. 활성화 함수는 인공 신경망의 활용 영역에 따라 달리 사용됩니다.



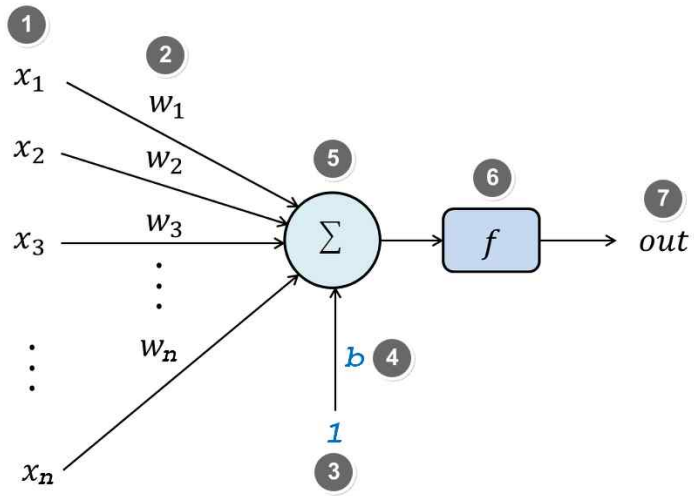
일반적으로 출력 값을 0에서 1사이의 값으로 하고자 할 경우엔 sigmoid 함수, 0보다 큰 출력 값만 내보내고자 할 경우엔 relu 함수를 사용합니다. 특히 다음은 분류를 위해 출력 층에 사용할 수 있는 활성화 함수로 softmax라고 합니다.



활성화 함수에 대해서는 뒤에서 자세히 살펴보도록 합니다. 여기서는 활성화 함수로 이러한 함수들이 사용된다는 정도로 이해하고 넘어갑니다.

인공 신경 함수 수식

다음은 하나의 인공 신경과 그 인공 신경으로 들어가는 ❶ 입력 값 x의 집합, ❷ 입력 값에 대한 가중치(신호 강도) w의 집합, ❸ 편향 입력 값 1, ❹ 편향 b, ❺ 가중치와 편향을 통해 들어오는 입력 값들의 합, ❻ 그 합을 입력으로 받는 활성화 함수 f, ❼ 활성화 함수 f의 출력 out을 나타냅니다.



인공 신경의 수식은 일반적으로 다음과 같습니다.

$$out = f(x_1 \times w_1 + x_2 \times w_2 + x_3 \times w_3 + \ldots + x_n \times w_n + 1 \times b)$$

$$out = f(\sum_{i=1}^{n} x_i \times w_i + 1 \times b)$$

예를 들어, 활성화 함수가 sigmoid 함수일 경우 인공 신경의 수식은 다음과 같습니다.

$$out = \frac{1}{1 + e^{-(x_1 \times w_1 + x_2 \times w_2 + x_3 \times w_3 + \ldots + x_n \times w_n + 1 \times b)}}$$

$$out = \frac{1}{1 + e^{-(\sum_{i=1}^{n} x_i \times w_i + 1 \times b)}}$$

또, 활성화 함수가 relu 함수일 경우 인공 신경의 수식은 다음과 같습니다.

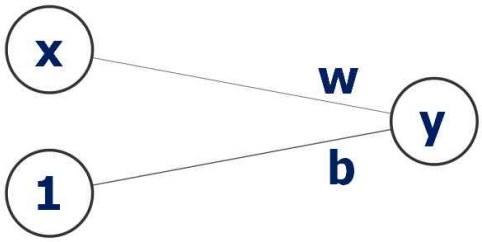
$$out = \max(0,\ x_1 \times w_1 + x_2 \times w_2 + x_3 \times w_3 + \ldots + x_n \times w_n + 1 \times b)$$

$$out = \max(0,\ \sum_{i=1}^{n} x_i \times w_i + 1 \times b)$$

이러한 수식들은 뒤에서 자세히 구현해 보면서 그 동작들을 이해합니다. 여기서는 개략적으로 살펴보고 넘어가도록 합니다.

가장 간단한 인공 신경

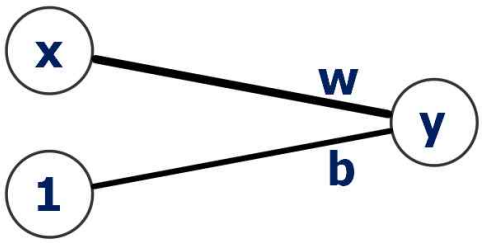
다음은 가장 간단한 형태의 인공 신경입니다.



이 인공 신경의 수식은 다음과 같습니다.

```
y = x*w + 1*b
```

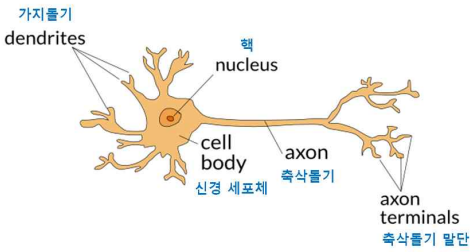
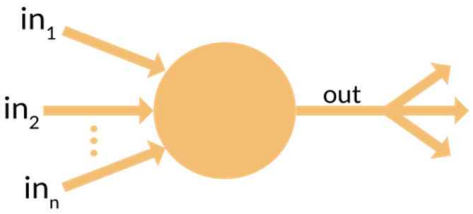
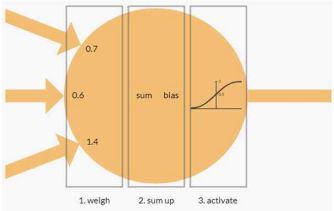
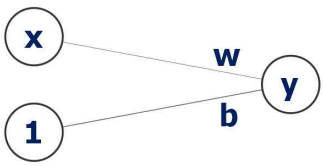
인공 신경의 학습과정은 가중치 w와 편향 b의 값을 조절하는 과정입니다. 인공 신경이 적절히 학습되면 입력값 x에 대해 원하는 출력값 y가 나오게 됩니다. 다음은 학습 과정을 거친 인공 신경입니다.



우리는 뒤에서 이 인공 신경을 직접 구현해보면서 인공 신경의 동작을 살펴봅니다.

이상에서 인간의 두뇌를 모델로 한 인공 신경망, 인공 신경망으로 할 수 있는 일들, 인공 신경망의 구조, 인공 신경망의 학습 방법, 생물학적 신경과 인공 신경과의 관계, 인공 신경의 구성 요소를 살펴보았습니다. 이 과정에서 인공 신경의 수식은 생물학적 신경으로부터 직관적으로 유도된 것을 알 수 있었습니다. 인공 신경의 수식은 간단한 형태의 수식이지만 이러한 인공 신경으로 망을 구성할 때는 아주 큰 힘을 발휘하게 됩니다.

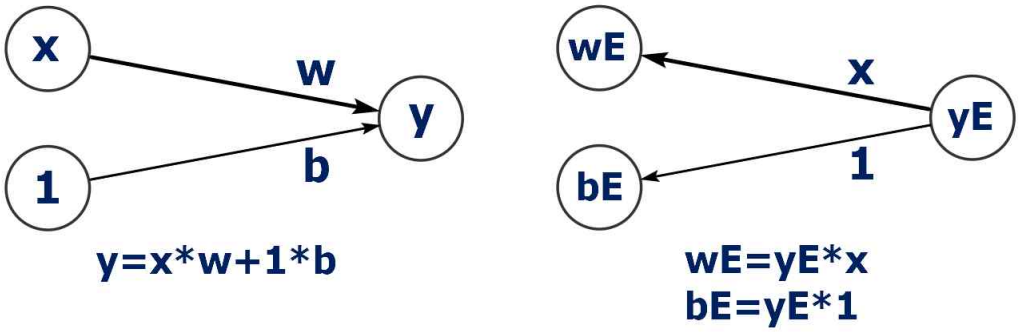
인공 신경망 기초 정리하기

1. 생체 신경	
2. 구조 단순화	
3. 동작 분석	
4. 단위 인공 신경망 구조	
5. 단위 인공 신경망 수식	$y = x*w + 1*b$

02 딥러닝 7 공식

여기서는 딥러닝 7 공식을 이용하여 딥러닝의 동작 원리를 이해해 봅니다. 또 딥러닝과 관련된 중요한 용어들, 예를 들어, 순전파, 목표값, 오차, 역전파 오차, 역전파, 학습률과 같은 용어들을 이해해 보도록 합니다.

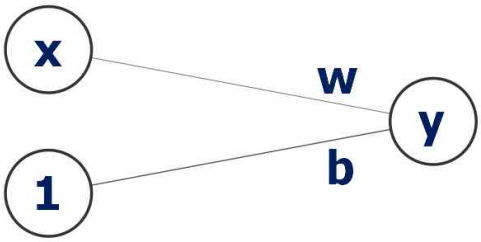
다음 그림은 이 단원에서 살펴볼 인공 신경의 순전파와 역전파를 나타내는 핵심 그림과 수식을 표현하고 있습니다.



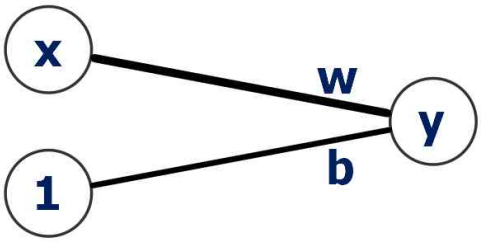
독자 여러분이 이 그림과 수식의 내용을 이해한다면 딥러닝의 가장 중요한 원리를 이해하게 되는 것입니다. 딥러닝의 핵심 원리 지금부터 이해해 봅시다!

01 딥러닝 제 1 공식 : 순전파

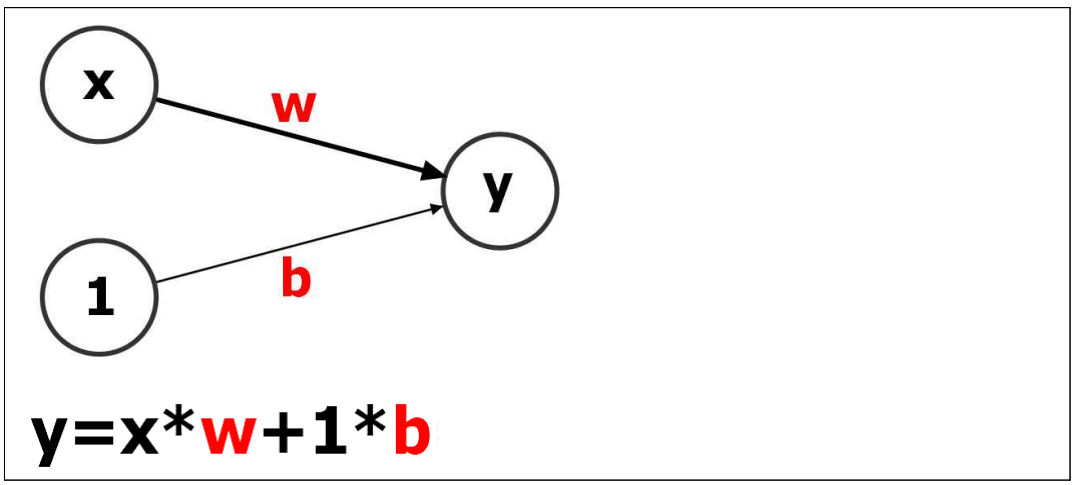
다음은 앞에서 소개한 단일 인공 신경의 그림입니다. 이 인공 신경은 입력 노드 1개, 출력 노드 1개, 편향으로 구성된 단일 인공 신경입니다.



이 인공 신경을 학습 시키면 다음과 같이 가중치와 편향의 값이 바뀌게 됩니다. 즉, 신호 전달 강도가 더 세지거나 약해집니다.



순전파 그림과 수식은 다음과 같습니다.



이 수식은 순전파 수식입니다. y는 예측값, x는 입력값, w는 가중치, b는 편향이라고 합니다.

순전파 살펴보기

이 수식에 대해서 구체적으로 생각해 봅니다. 다음과 같이 각 변수에 값을 줍니다.

```
x = 2
w = 3
b = 1
```

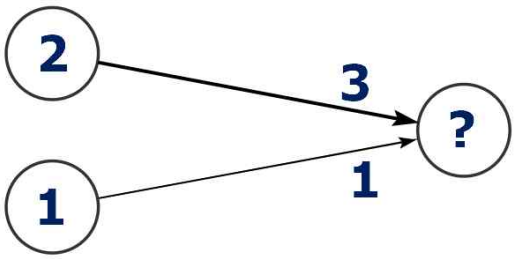
그러면 식은 다음과 같이 됩니다.

```
y = 2*3 + 1*1
y = ?
```

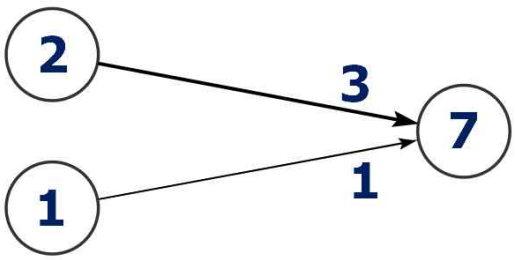
y는 얼마가 될까요? 다음과 같이 계산해서 y는 7이 됩니다.

```
2*3 + 1*1 = 7
```

이 상황을 그림으로 생각해 봅니다. 다음과 같이 x, w, b 값이 y로 흘러가는 인공 신경이 있습니다. 이 과정을 인공 신경의 순전파라고 합니다.



이 경우 y로 얼마가 나올까요? 앞에서 살펴본 대로 다음과 같이 7이 나오게 됩니다.



여기서 순전파를 통해 나온 7을 예측값이라고 합니다.

순전파 코딩하기

돌다리도 두들겨 보고 건넌다고 순전파를 아두이노 스케치로 구현해 봅니다.

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
321.ino

```
double x = 2; // 일반적으로 딥러닝 변수는 실수입니다.
double w = 3;
double b = 1;

void setup() {

        Serial.begin(115200);

        double y = x*w + 1*b; // 딥러닝 제 1 공식 : 순전파
        Serial.print("y : "); Serial.println(y, 6); // 소수점 이하 6자리 출력

}

void loop() {
}
```

예제를 아두이노 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
y : 7.000000
```

02 딥러닝 제 2 공식 : 평균 제곱 오차

다음은 딥러닝 제 2 공식을 나타냅니다.

E = (y-yT)*(y-yT) / 2

이 수식은 평균 제곱 오차 수식이라고 합니다. E는 오차, y는 순전파에 의한 예측값, yT는 목표값 또는 라벨을 나타냅니다. yT는 입력값 x에 대해 실제로 나오기를 원하는 값입니다. 오차(error)는 손실(loss) 또는 비용(cost)이라고도 합니다. 오차값이 작을수록 예측을 잘하는 인공 신경망입니다.

앞에서 우리는 y값으로 7을 얻었습니다. 그런데 y로 10이 나오게 하고 싶습니다. 이 경우 yT 값은 10이 됩니다. 그러면 평균 제곱 오차는 다음과 같이 계산됩니다.

```
E = (7 - 10)*(7 - 10)/2 = (-3)*(-3)/2 = 9/2 = 4.5
```

평균 제곱 오차 코딩하기

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
322.ino

```
double x = 2; // 일반적으로 딥러닝 변수는 실수입니다.
double w = 3;
double b = 1;
double yT = 10; // 목표값을 10이라고 가정합니다.

void setup() {

        Serial.begin(115200);

        double y = x*w + 1*b; // 딥러닝 제 1 공식 : 순전파
        double E = (y - yT)*(y - yT) / 2; // 딥러닝 제 2 공식 : 평균 제곱 오차
        Serial.print("E : "); Serial.println(E, 6);

}

void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
E : 4.500000
```

03 딥러닝 제 3 공식 : 역전파 오차

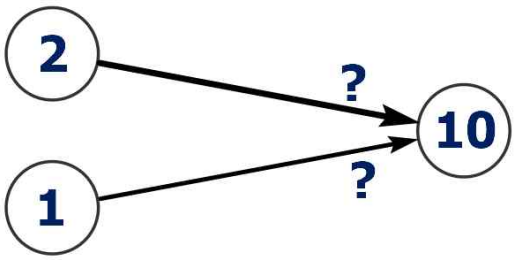
다음은 딥러닝 제 3 공식을 나타냅니다.

yE = y - yT

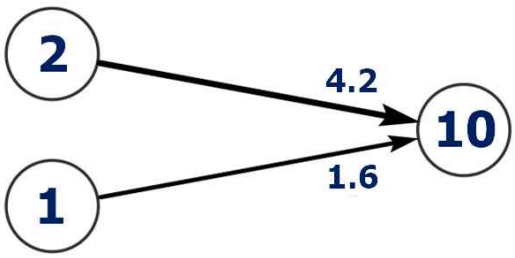
yE는 역전파 오차, y는 순전파에 의한 예측값, yT는 목표값 또는 라벨을 나타냅니다. yT는 입력값에 대해 실제로 나오기를 원하는 값입니다. 역전파 오차를 구하는 수식은 단순합니다. 예측값 y에서 목표값 yT를 뺀 값이 역전파 오차가 됩니다. yE 값이 다음 단계에서 역전파되기 때문에 역전파 오차라고 합니다. yE의 정확한 의미는 y에 대한 오차 E의 순간변화율을 의미하며 편미분을 통해 유도됩니다.

딥러닝 제 3 공식에 대해서 구체적으로 살펴봅니다.

앞에서 우리는 y값으로 7을 얻었습니다. 그런데 y로 10이 나오게 하고 싶습니다. 이런 경우에 w와 b의 값을 어떻게 바꿔야 할까요?



y값이 10이 되려면 3이 모자랍니다. y의 오차는 w와 b의 오차로 인해 발생합니다. 따라서 w와 b값을 적당히 증가시키면 y로 10에 가까운 값이 나오게 할 수 있겠죠? 예를 들어, w를 4.2로, b를 1.6으로 증가시키면 y의 값은 10이 됩니다.



그러면 w, b값을 어떤 기준으로 얼마나 증가시켜야 할까요? 이 과정을 이해하는 것이 바로 역전파의 핵심을 이해하는 것이고 나아가서 딥러닝의 핵심을 이해하는 것입니다. 이 과정을 자세히 살펴봅니다.

이전 그림에서 y로 7이 흘러나갔는데 우리는 이 값이 10이 되기를 원합니다. 여기서 10은 목표값이 됩니다. 참고로 인공지능 학습 시 목표값은 라벨이라고 합니다. 다음 수식에서 yT는 목표값을 나타내며 10을 갖습니다.

```
yT = 10
y = 7
yE = y - yT = -3
```

y값은 현재값 7인 상태이며, yE는 현재값에서 목표값을 뺀 값 -3이 됩니다. 이 때, yE값을 역전파 오차라고 하며, 역전파에 사용할 오차값입니다.

역전파 오차 코딩하기

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
323.ino

```
double x = 2; // 일반적으로 딥러닝 변수는 실수입니다.
double w = 3;
double b = 1;
double yT = 10; // 목표값을 10이라고 가정합니다.

void setup() {

        Serial.begin(115200);

        double y = x*w + 1*b; // 딥러닝 제 1 공식 : 순전파
        double E = (y - yT)*(y - yT) / 2; // 딥러닝 제 2 공식 : 평균 제곱 오차
        double yE = y - yT; // 딥러닝 제 3 공식 : 역전파 오차
        Serial.print("yE : "); Serial.println(yE, 6);

}

void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
yE : -3.000000
```

04 딥러닝 제 4 공식 : 입력 역전파

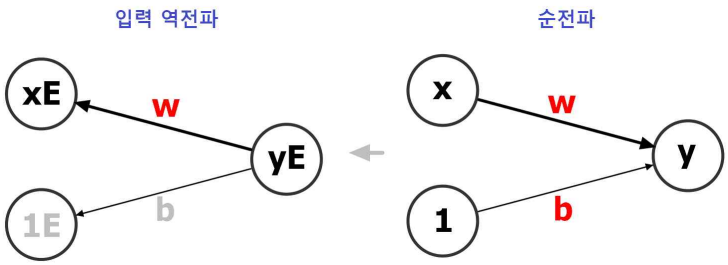
다음은 딥러닝 제 4 공식을 나타냅니다.



xE는 입력 역전파, yE는 역전파 오차로 딥러닝 제 3 공식에서 구한 값입니다. 회색으로 표시된 1E는 숫자 1의 오차라는 의미로 사용하지 않는 부분입니다. 딥러닝 제 4 공식은 다음과 같은 순서로 유도할 수 있습니다.

1. 딥러닝 제 1 공식의 그림을 복사합니다.
2. y -> yE, x -> xE로 변경합니다.
3. 화살표 방향을 반대로 합니다.
4. 1E, b는 사용하지 않습니다.

다음 그림을 참조합니다.



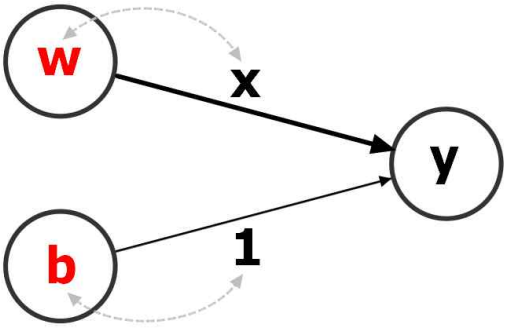
이 수식은 출력층에서 발생한 역전파 오차가 입력층으로 흘러가는 상황을 나타냅니다. 딥러닝 제 4 공식은 실제로 은닉층으로 전달되는 역전파에 사용하는 공식으로 뒤에서 사용합니다. xE의 정확한 의미는 x에 대한 오차 E의 순간변화율을 의미하며 편미분을 통해 유도됩니다.

05 딥러닝 제 5 공식 : 가중치, 편향 순전파

다음은 가중치와 편향의 순전파를 나타내는 식과 그림입니다.

y=x*w+1*b (딥러닝 제 1 공식)

y=w*x+b*1 (딥러닝 제 5 공식)

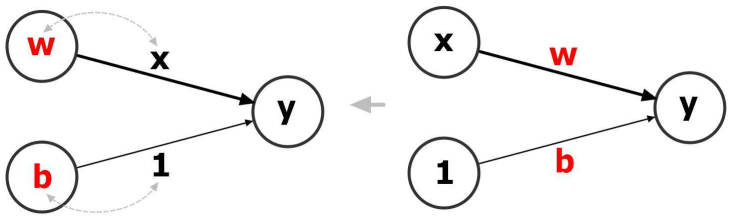


(가중치, 편향 순전파 그림)

딥러닝 제 5 공식은 다음과 같은 순서로 유도할 수 있습니다.

1. 딥러닝 제 1 공식을 복사합니다.
2. x와 w, 1과 b를 교환하여 딥러닝 제 5 공식을 유도합니다.
3. 딥러닝 제 1 공식의 그림과 같은 형태로 그림을 그립니다.

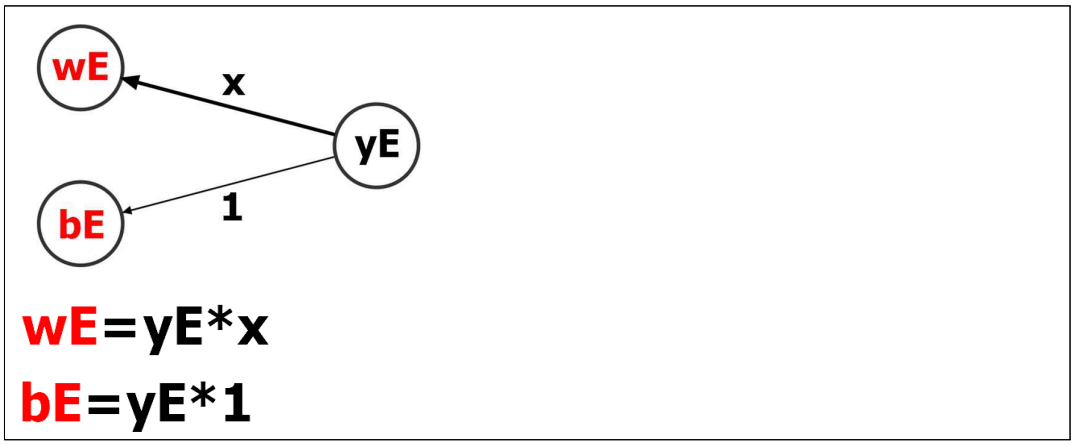
다음 그림을 참조합니다.



딥러닝 제 5 공식은 다음에 나올 딥러닝 제 6 공식을 유도하기 위해 사용하며, 실제로 딥러닝 제 1 공식과 같습니다. 딥러닝 제 5 공식은 구현을 위해 사용하지는 않습니다.

06 딥러닝 제 6 공식 : 가중치, 편향 역전파

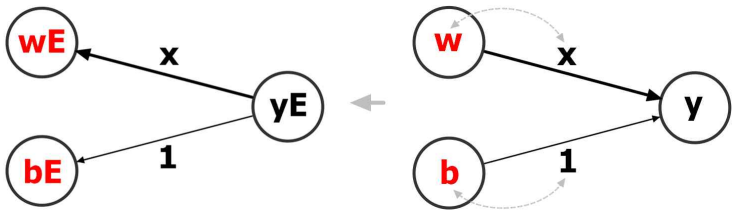
다음은 가중치와 편향의 역전파를 나타내는 그림과 수식입니다.



wE는 가중치 역전파 오차, bE는 편향 역전파 오차를 의미합니다. yE는 역전파 오차로 딥러닝 제 3 공식에서 구한 값입니다. 딥러닝 제 6 공식은 다음과 같은 순서로 유도할 수 있습니다.

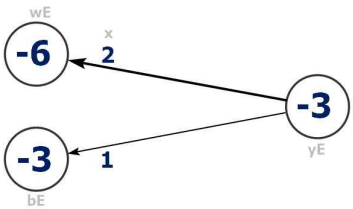
1. 딥러닝 제 5 공식의 그림을 복사합니다.
2. y -> yE, w -> wE, b -> bE로 변경합니다.
3. 화살표 방향을 반대로 합니다.

다음 그림을 참조합니다.



딥러닝 제 6 공식은 실제로 편미분을 연쇄적으로 적용하여 얻은 공식입니다. 편미분을 이용하여 역전파를 유도하는 방법은 부록을 참조합니다. wE의 정확한 의미는 w에 대한 오차 E의 순간변화율을 의미합니다. 마찬가지로 bE는 b에 대한 오차 E의 순간변화율을 의미합니다.

위 수식에 의해 wE, bE는 다음 그림과 같이 계산됩니다.



딥러닝 제 3 공식의 예에서 우리는 yE값으로 -3을 얻었습니다.

가중치, 편향 역전파 코딩하기

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
326.ino

```
double x = 2; // 일반적으로 딥러닝 변수는 실수입니다.
double w = 3;
double b = 1;
double yT = 10; // 목표값을 10이라고 가정합니다.

void setup() {

        Serial.begin(115200);

        double y = x*w + 1*b; // 딥러닝 제 1 공식 : 순전파
        double E = (y - yT)*(y - yT) / 2; // 딥러닝 제 2 공식 : 평균 제곱 오차
        double yE = y - yT; // 딥러닝 제 3 공식 : 역전파 오차
        double wE = yE*x; // 딥러닝 제 6 공식 : 가중치 역전파
        double bE = yE*1; // 딥러닝 제 6 공식 : 편향 역전파
        Serial.print("wE : "); Serial.println(wE, 6);
        Serial.print("bE : "); Serial.println(bE, 6);

}

void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
wE : -6.000000
bE : -3.000000
```

07 딥러닝 제 7 공식 : 신경망 학습

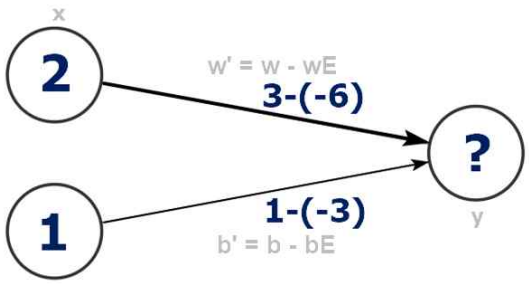
다음은 신경망 학습을 나타내는 수식입니다. 신경망은 가중치를 의미합니다.

```
w -= lr*wE
 b -= lr*bE
```

lr은 learning rate의 약자로 학습률을 의미합니다. wE는 가중치 역전파 오차, bE는 편향 역전파 오차로 딥러닝 제 6 공식에서 구한 값입니다. 딥러닝 제 7 공식은 실제로 경사하강법이라고 하며 미분을 이용하여 얻은 공식입니다. 이 수식을 적용하여 w, b 값을 갱신하는 과정을 인공 신경망의 학습이라고 합니다.

신경망 학습해 보기

앞에서 우리는 wE, bE 값으로 각각 -6, -3을 구하였습니다. wE, bE 값을 다음 그림과 같이 적용해 봅니다. 다음 그림에서 w는 이전 가중치, w'는 갱신된 가중치를 의미합니다. 그리고 앞에서 구한 wE, bE의 값을 각각 w, b에서 빼줍니다. 그러면 갱신된 w', b' 값이 증가하게 됩니다. 이 부분을 좀 더 자세히 살펴봅시다. 앞에서 살펴본 딥러닝 제 3 공식을 변형하면 y - yE = yT가 되어 y에서 yE를 빼면 yT가 됩니다. 즉, y값 7에서 yE값 -3을 빼주면 yT값 10이 됩니다. 같은 원리로 w에서 wE를, b에서 bE를 빼주도록 합니다.



그런데 wE, bE값이 너무 큽니다. 이 상태로 계산을 하면 새로운 y값은 (2*9+1*4)와 같이 계산되어 22가 되게 되며, 우리가 원하는 목표값 10보다 더 큰 값이 나오게 됩니다. 구체적인 계산 과정은 다음과 같습니다.

```
x = 2
w' = w - wE = 3-(-6) = 9
b' = b - bE = 1-(-3) = 4
y = x*w' + 1*b' = 2*9+1*4 = 22
```

신경망 학습 코딩하기 1

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
327.ino

```
double x = 2; // 일반적으로 딥러닝 변수는 실수입니다.
double w = 3;
double b = 1;
double yT = 10; // 목표값을 10이라고 가정합니다.
double lr  = 0.01;

void setup() {

        Serial.begin(115200);

        double y = x*w + 1*b; // 딥러닝 제 1 공식 : 순전파
        double E = (y - yT)*(y - yT) / 2; // 딥러닝 제 2 공식 : 평균 제곱 오차
        double yE = y - yT; // 딥러닝 제 3 공식 : 역전파 오차
        double wE = yE*x; // 딥러닝 제 6 공식 : 가중치 역전파
        double bE = yE*1; // 딥러닝 제 6 공식 : 편향 역전파
        w -= wE;
        b -= bE;
        y = x*w + 1*b;
        Serial.print("w : "); Serial.println(w, 6);
        Serial.print("b : "); Serial.println(b, 6);
        Serial.print("y : "); Serial.println(y, 6);

}

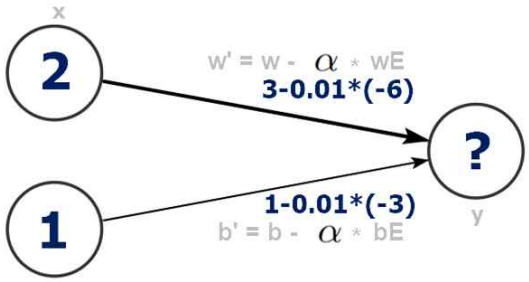
void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
w : 9.000000
b : 4.000000
y : 22.000000
```

학습률 적용하기

그래서 나온 방법이 학습률입니다. wE, bE에 적당히 작은 값을 곱해주어 값을 줄이는 겁니다. 여기서는 0.01을 곱해줍니다. 그러면 다음과 같이 계산할 수 있습니다.



*** 여기서 α는 학습률이라고 하며 뒤에서는 lr이라는 이름으로 구현합니다. lr은 learning rate의 약자로 학습률을 의미합니다.

이렇게 하면 2*(3.06)+1.03=7.15가 나옵니다. 이렇게 조금씩 늘려나가면 10을 만들 수 있습니다.

여기서 곱해준 0.01은 학습률이라고 하는 값입니다. 일반적으로 학습률 값은 0.001로 시작하여 학습이 진행되는 상황에 따라 조금씩 늘이거나 줄여서 사용합니다.

신경망 학습 코딩하기 2

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
327_2.ino

```
double x = 2; // 일반적으로 딥러닝 변수는 실수입니다.
double w = 3;
double b = 1;
double yT = 10; // 목표값을 10이라고 가정합니다.
double lr  = 0.01;

void setup() {

        Serial.begin(115200);

        double y = x*w + 1*b; // 딥러닝 제 1 공식 : 순전파
        double E = (y - yT)*(y - yT) / 2; // 딥러닝 제 2 공식 : 평균 제곱 오차
        double yE = y - yT; // 딥러닝 제 3 공식 : 역전파 오차
        double wE = yE*x; // 딥러닝 제 6 공식 : 가중치 역전파
        double bE = yE*1; // 딥러닝 제 6 공식 : 편향 역전파
        w -= lr*wE; // 딥러닝 제 7 공식 : 가중치 학습
        b -= lr*bE; // 딥러닝 제 7 공식 : 편향 학습
        Serial.print("w : "); Serial.println(w, 6);
        Serial.print("b : "); Serial.println(b, 6);

}

void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
w : 3.060000
b : 1.030000
```

학습된 신경망으로 예측하기

인공 신경망에 대해 딥러닝 제 1 공식~딥러닝 제 7 공식까지 1회 수행을 하면 한 번의 학습이 이루어지게 됩니다. 여기서는 1회 학습된 신경망으로 y값을 예측해 봅니다.

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
327_3.ino

```
double x = 2; // 일반적으로 딥러닝 변수는 실수입니다.
double w = 3;
double b = 1;
double yT = 10; // 목표값을 10이라고 가정합니다.
double lr  = 0.01;

void setup() {

        Serial.begin(115200);

        double y = x*w + 1*b; // 딥러닝 제 1 공식 : 순전파
        double E = (y - yT)*(y - yT) / 2; // 딥러닝 제 2 공식 : 평균 제곱 오차
        double yE = y - yT; // 딥러닝 제 3 공식 : 역전파 오차
        double wE = yE*x; // 딥러닝 제 6 공식 : 가중치 역전파
        double bE = yE*1; // 딥러닝 제 6 공식 : 편향 역전파
        w -= lr*wE; // 딥러닝 제 7 공식 : 가중치 학습
        b -= lr*bE; // 딥러닝 제 7 공식 : 편향 학습

        y = x*w + 1*b; // 1회 학습 후 예측값
        Serial.print("y : "); Serial.println(y, 6);

}

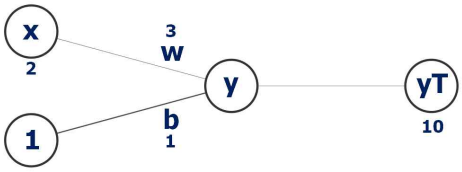
void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
y : 7.150000
```

08 딥러닝 반복 학습해 보기

지금까지의 과정을 반복해서 학습해 봅니다. 다음 그림을 살펴봅니다.



이 그림에서 입력값 x, 가중치 w, 편향 b는 각각 2, 3, 1이고 목표값 yT는 10입니다. x, yT를 이용하여 w, b에 대해 학습을 수행해 봅니다.

*** 이 값들은 임의의 값들입니다. 다른 값들을 사용하여 학습을 수행할 수도 있습니다.

반복 학습 2회 수행하기

여기서는 반복 학습 2회를 수행해 봅니다.

1. 다음과 같이 이전 예제를 수정합니다.
328.ino

```
double x = 2;
double w = 3;
double b = 1;
double yT = 10;
double lr  = 0.01;

void setup() {

        Serial.begin(115200);

        for(int epoch=0;epoch<2;epoch++) {

                double y = x*w + 1*b;
                double E = (y - yT)*(y - yT) / 2;
                double yE = y - yT;
                double wE = yE*x;
                double bE = yE*1;
                w -= lr*wE;
                b -= lr*bE;

                Serial.print("epoch = "); Serial.println(epoch);
                Serial.print("y : "); Serial.println(y, 3);
                Serial.print("w : "); Serial.println(w, 3);
```

```
                    Serial.print("b : "); Serial.println(b, 3);

            }

}

void loop() {
}
```

09 : epoch값을 0에서 2 미만까지 바꾸어가며 11~24줄을 2회 수행합니다.
19 : printf 함수를 호출하여 epoch 값을 출력해 줍니다.

2. 

업로드를 수행합니다. 다음은 실행 결과 화면입니다.

```
epoch = 0
y : 7.000
w : 3.060
b : 1.030
epoch = 1
y : 7.150
w : 3.117
b : 1.058
```

y 값이 7에서 7.150으로 바뀌는 것을 확인합니다. wE, bE 값을 확인합니다. 또, w, b 값을 확인합니다. w, b의 값이 아주 조금 증가한 것을 확인합니다. 가지 돌기의 두께가 조금 두꺼워진 것과 같습니다.

반복 학습 20회 수행하기

여기서는 반복 학습 20회를 수행해 봅니다.

1. 다음과 같이 예제를 수정합니다.
328_2.ino

```
        for(int epoch=0;epoch<20;epoch++) {
```

09 : epoch값을 0에서 20 미만까지 수행합니다.

2.  업로드를 수행합니다. 다음은 실행 결과 화면입니다.

```
epoch = 18
y : 8.808
w : 3.747
b : 1.374
epoch = 19
y : 8.868
w : 3.770
b : 1.385
```

y 값이 8.868까지 접근하는 것을 확인합니다. w, b의 값이 처음에 비해 증가한 것을 확인합니다. 가지돌기의 두께가 더 두꺼워진 것과 같습니다.

반복 학습 200회 수행하기

여기서는 반복 학습 200회를 수행해 봅니다.

1. 다음과 같이 예제를 수정합니다.
328_3.ino

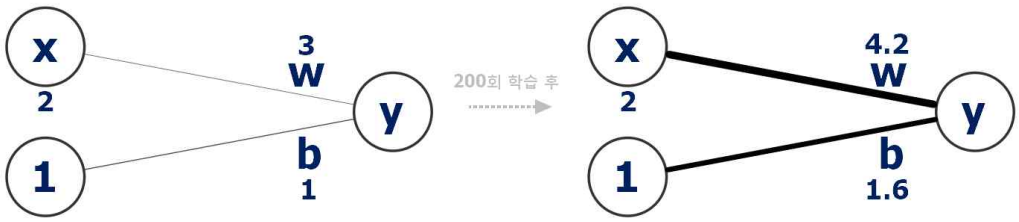
```
for(int epoch=0;epoch<200;epoch++) {
```

09 : epoch값을 0에서 200 미만까지 수행합니다.

2. 업로드를 수행합니다. 다음은 실행 결과 화면입니다.

```
epoch = 198
y : 10.000
w : 4.200
b : 1.600
epoch = 199
y : 10.000
w : 4.200
b : 1.600
```

y 값이 10.000에 수렴하는 것을 확인합니다. 이 때, 가중치 w는 4.2, 편향 b는 1.6에 수렴합니다.



200회 학습 후에는 그림처럼 w, b의 강도가 세지는 것을 볼 수 있습니다. 인공 신경망의 학습은 이처럼 가중치 w, 편향 b의 값을 조정하는 과정입니다.

오차 조건 추가하기

여기서는 오차값이 충분히 작아지면 학습을 자동으로 중단하도록 해 봅니다.

1. 다음과 같이 이전 예제를 수정합니다.
328_4.ino

```
double x = 2;
double w = 3;
double b = 1;
double yT = 10;
double lr  = 0.01;

void setup() {

        Serial.begin(115200);

        for(int epoch=0;epoch<200;epoch++) {

                double y = x*w + 1*b;
                double E = (y - yT)*(y - yT) / 2;
                double yE = y - yT;
                double wE = yE*x;
                double bE = yE*1;
                w -= lr*wE;
                b -= lr*bE;

                Serial.print("epoch = "); Serial.println(epoch);
                Serial.print("y : "); Serial.println(y, 3);
                Serial.print("w : "); Serial.println(w, 3);
                Serial.print("b : "); Serial.println(b, 3);

                if(E< 0.0000001) break;

        }

}

void loop() {
}
```

24 : 오차값 E가 0.0000001(1천만분의1)보다 작으면 break문을 수행하여 09줄의 for문을 빠져나갑니다.

2.  업로드를 수행합니다. 다음은 실행 결과 화면입니다.

```
epoch = 171
y : 10.000
w : 4.200
b : 1.600
epoch = 172
y : 10.000
w : 4.200
b : 1.600
```

epoch 값이 172일 때 for 문을 빠져 나갑니다. y값은 10에 수렴합니다.

학습률 변경하기

여기서는 학습률 값을 변경시켜 보면서 학습의 상태를 살펴봅니다.

1. 다음과 같이 예제를 수정합니다.
328_5.ino

```
double lr   = 0.05;
```

05 : 학습률 값을 0.05로 변경합니다.

2.  업로드를 수행합니다. 다음은 실행 결과 화면입니다.

```
epoch = 30
y : 9.999
w : 4.200
b : 1.600
epoch = 31
y : 10.000
w : 4.200
b : 1.600
```

epoch 값이 31일 때 학습이 완료되는 것을 볼 수 있습니다.

3. 다음과 같이 예제를 수정합니다.
328_6.ino

```
double lr   = 0.005;
```

05 : 학습률 값을 0.005로 변경합니다.

4.  업로드를 수행합니다. 다음은 실행 결과 화면입니다.

```
epoch = 198
y : 9.980
w : 4.192
b : 1.596
epoch = 199
y : 9.981
w : 4.192
b : 1.596
```

epoch 값이 199일 때 학습이 완료되지 않은 상태로 종료되는 것을 볼 수 있습니다.

5. 다음과 같이 예제를 수정합니다.
328_7.ino

```
        for(int epoch=0;epoch<2000;epoch++) {
```

09 : epoch값을 0에서 2000 미만까지 수행합니다.

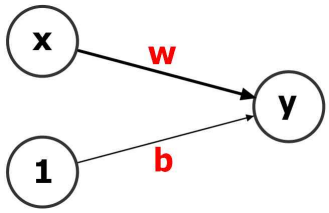
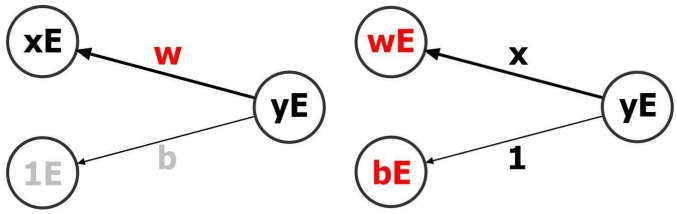
6.  업로드를 수행합니다. 다음은 실행 결과 화면입니다.

```
epoch = 348
y : 10.000
w : 4.200
b : 1.600
epoch = 349
y : 10.000
w : 4.200
b : 1.600
```

epoch 값이 349일 때 학습이 완료되는 것을 볼 수 있습니다.

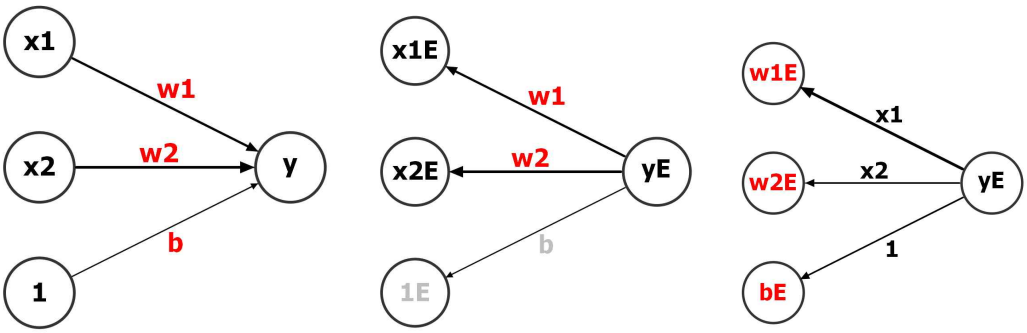
09 딥러닝 7 공식 정리하기

다음은 순전파, 역전파를 나타내는 신경망의 동작입니다.

순전파	역전파
	

03 딥러닝 7 공식 확장하기

지금까지 우리는 가장 간단한 형태인 [1입력 1출력] 인공 신경에 대해 딥러닝 7 공식을 살펴보고 구현해 보았습니다. 여기서는 [2입력 1출력] 인공 신경과 [2입력 2출력] 인공 신경망으로 딥러닝 7 공식을 확장해 봅니다.

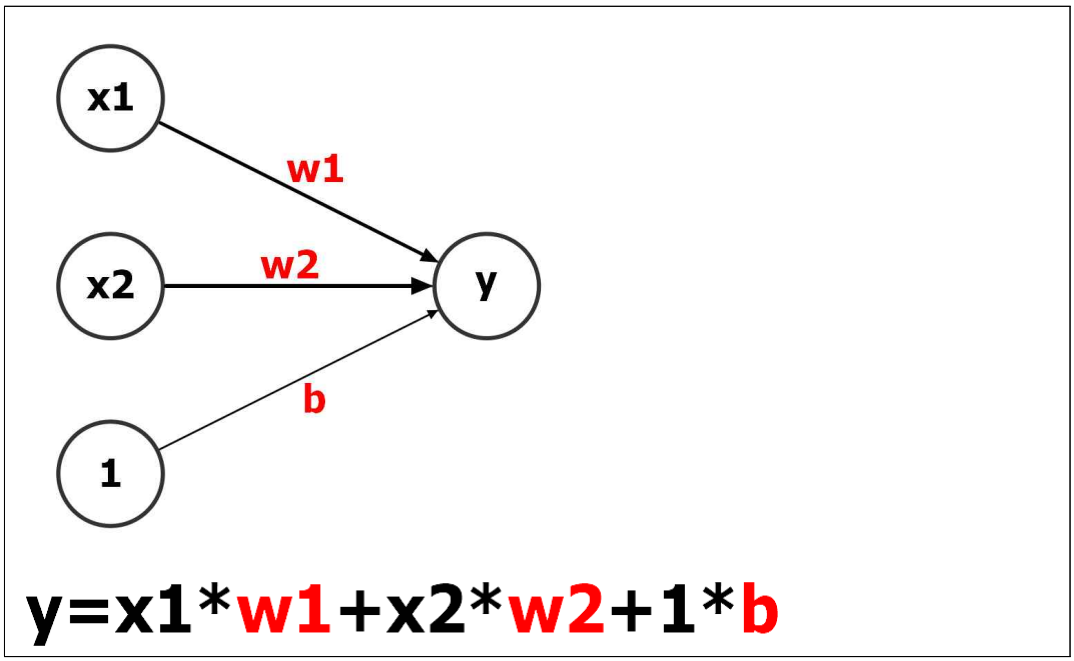


01 2입력 1출력 인공 신경

먼저 [2입력 1출력] 인공 신경에 대해 딥러닝 7 공식을 확장하고 구현해 봅니다.

딥러닝 제 1 공식 : 순전파

다음은 [2입력 1출력] 인공 신경의 딥러닝 제 1 공식을 나타냅니다.



이 수식은 순전파 수식입니다. y는 예측값, x1, x2는 입력값, w1, w2는 가중치, b는 편향입니다. 앞에서 살펴본 [1입력 1출력] 인공 신경에 입력과 가중치가 하나씩 추가되었습니다.

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
331.ino

```
double x1 = 2, x2 = 3;
double w1 = 3, w2 = 4;
double b = 1;

void setup() {

        Serial.begin(115200);

        double y = x1*w1 + x2*w2 + 1*b;
        Serial.print("y : "); Serial.println(y, 6);

}

void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
y : 19.000000
```

딥러닝 제 2 공식 : 평균 제곱 오차

다음은 [2입력 1출력] 인공 신경의 딥러닝 제 2 공식을 나타냅니다.

$$E = (y-yT)*(y-yT) / 2$$

[2입력 1출력] 인공 신경은 [1입력 1출력] 인공 신경과 출력의 개수가 같기 때문에 평균 제곱 오차 수식이 같습니다.

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
331_2.ino

```
double x1 = 2, x2 = 3;
double w1 = 3, w2 = 4;
double b = 1;
double yT = 27;

void setup() {

        Serial.begin(115200);

        double y = x1*w1 + x2*w2 + 1*b;
        double E = (y - yT)*(y - yT) / 2;
        Serial.print("E : "); Serial.println(E, 6);

}

void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
E : 32.000000
```

딥러닝 제 3 공식 : 역전파 오차

다음은 [2입력 1출력] 인공 신경의 딥러닝 제 3 공식을 나타냅니다.

$$yE = y - yT$$

[2입력 1출력] 인공 신경은 [1입력 1출력] 인공 신경과 출력의 개수가 같기 때문에 역전파 오차 수식이 같습니다.

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
331_3.ino

```
double x1 = 2, x2 = 3;
double w1 = 3, w2 = 4;
double b = 1;
double yT = 27;

void setup() {

        Serial.begin(115200);

        double y = x1*w1 + x2*w2 + 1*b;
        double E = (y - yT)*(y - yT) / 2;
        double yE = y - yT;
        Serial.print("yE : "); Serial.println(yE, 6);

}

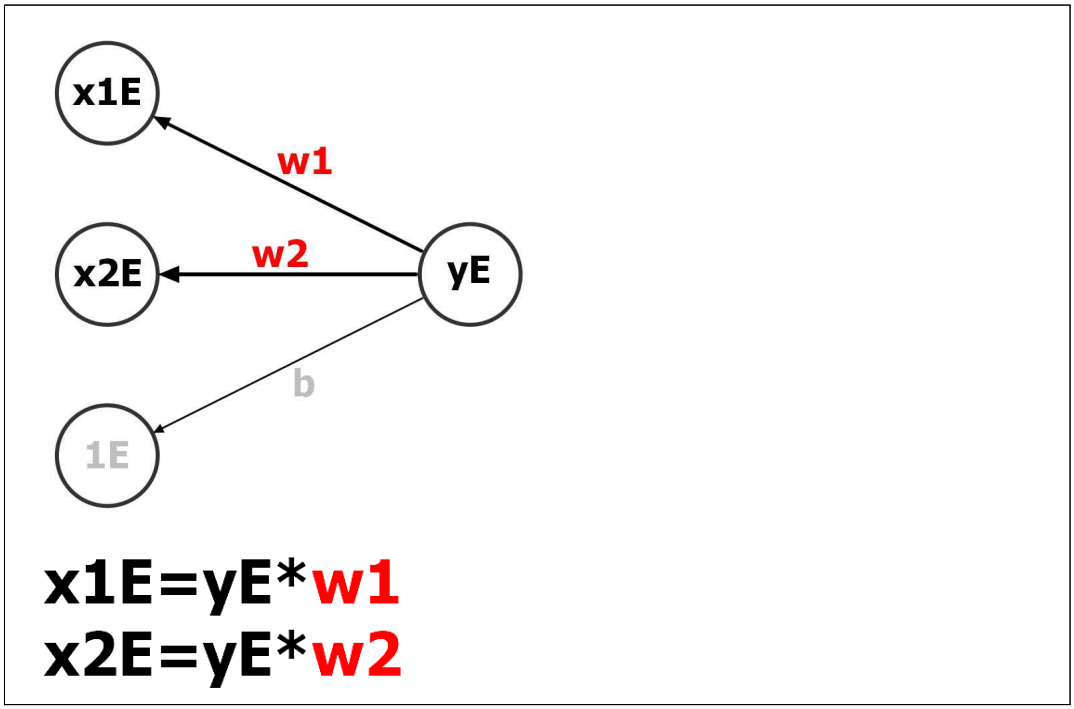
void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
yE : -8.000000
```

딥러닝 제 4 공식 : 입력 역전파

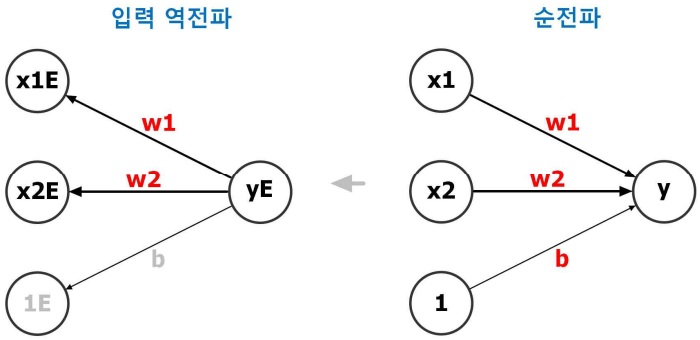
다음은 [2입력 1출력] 인공 신경의 딥러닝 제 4 공식을 나타냅니다.



x1E, x2E는 입력 역전파, yE는 역전파 오차로 딥러닝 제 3 공식에서 구한 값입니다. 회색으로 표시된 1E는 숫자 1의 오차라는 의미로 사용하지 않는 부분입니다. 딥러닝 제 4 공식은 다음과 같은 순서로 유도할 수 있습니다.

1. 딥러닝 제 1 공식의 그림을 복사합니다.
2. y -> yE, x1 -> x1E, x2 -> x2E로 변경합니다.
3. 화살표 방향을 반대로 합니다.
4. 1E, b는 사용하지 않습니다.

다음 그림을 참조합니다.



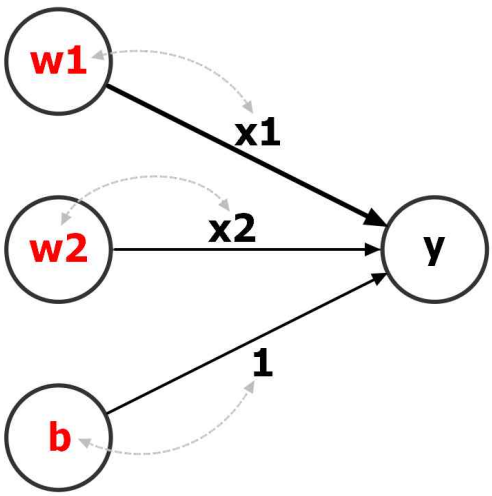
이 수식은 출력층에서 발생한 역전파 오차가 입력층으로 흘러가는 상황을 나타냅니다. 딥러닝 제 4 공식은 실제로 은닉층으로 전달되는 역전파에 사용하는 공식으로 뒤에서 사용합니다.

05 딥러닝 제 5 공식 : 가중치, 편향 순전파

다음은 [2입력 1출력] 인공 신경의 가중치와 편향의 순전파를 나타내는 식과 그림입니다.

y=x1*w1+x2*w2+1*b (딥러닝 제 1 공식)

y=w1*x1+w2*x2+b*1 (딥러닝 제 5 공식)

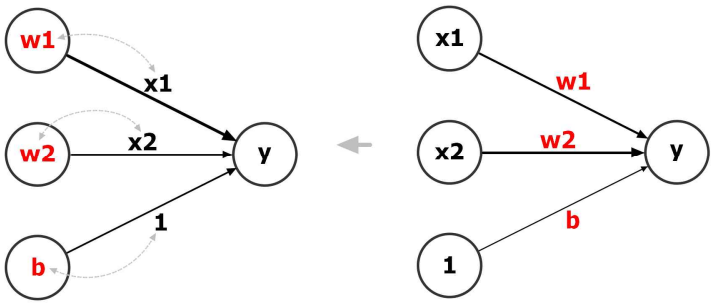


(가중치, 편향 순전파 그림)

딥러닝 제 5 공식은 다음과 같은 순서로 유도할 수 있습니다.

1. 딥러닝 제 1 공식을 복사합니다.
2. x1과 w1, x2와 w2, 1과 b를 교환하여 딥러닝 제 5 공식을 유도합니다.
3. 딥러닝 제 1 공식의 그림과 같은 형태로 그림을 그립니다.

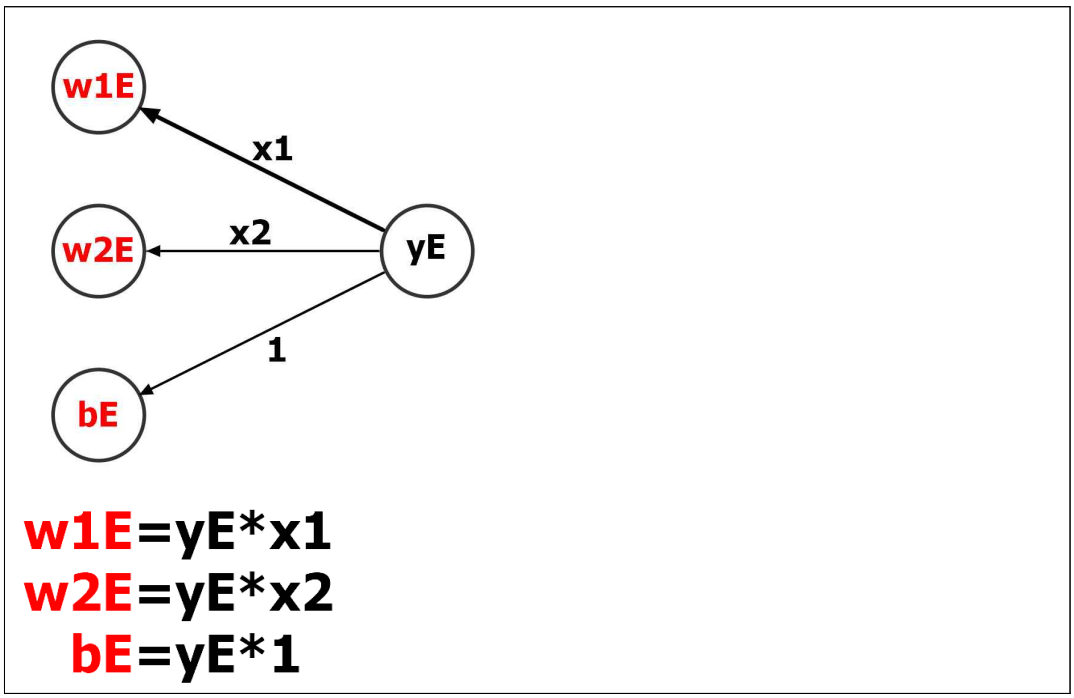
다음 그림을 참조합니다.



딥러닝 제 5 공식은 다음에 나올 딥러닝 제 6 공식을 유도하기 위해 사용하며, 실제로 딥러닝 제 1 공식과 같습니다. 딥러닝 제 5 공식은 구현을 위해 사용하지는 않습니다.

딥러닝 제 6 공식 : 가중치, 편향 역전파

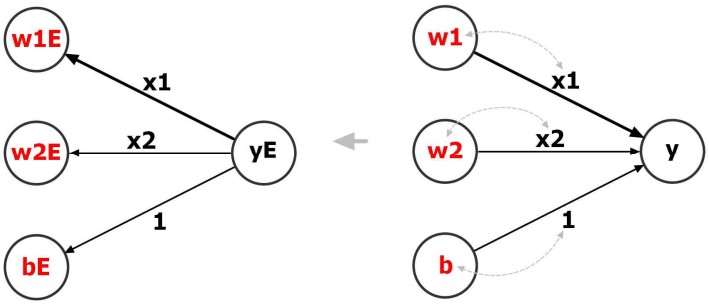
다음은 [2입력 1출력] 인공 신경의 가중치와 편향의 역전파를 나타내는 그림과 수식입니다.



w1E, w2E는 가중치 역전파 오차, bE는 편향 역전파 오차를 의미합니다. yE는 역전파 오차로 딥러닝 제 3 공식에서 구한 값입니다. 딥러닝 제 6 공식은 다음과 같은 순서로 유도할 수 있습니다.

1. 딥러닝 제 5 공식의 그림을 복사합니다.
2. y -> yE, w1 -> w1E, w2 -> w2E, b -> bE로 변경합니다.
3. 화살표 방향을 반대로 합니다.

다음 그림을 참조합니다.



딥러닝 제 6 공식은 실제로 편미분을 연쇄적으로 적용하여 얻은 공식입니다. 편미분을 이용하여 역전파를 유도하는 방법은 부록을 참조합니다.

다다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
331_4.ino

```
double x1 = 2, x2 = 3;
double w1 = 3, w2 = 4;
double b = 1;
double yT = 27;

void setup() {

        Serial.begin(115200);

        double y = x1*w1 + x2*w2 + 1*b;
        double E = (y - yT)*(y - yT) / 2;
        double yE = y - yT;
        double w1E = yE*x1;
        double w2E = yE*x2;
        double bE = yE*1;
        Serial.print("w1E : "); Serial.println(w1E, 6);
        Serial.print("w2E : "); Serial.println(w2E, 6);
        Serial.print("bE : "); Serial.println(bE, 6);

}

void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
w1E : -16.000000
w2E : -24.000000
bE : -8.000000
```

07 딥러닝 제 7 공식 : 신경망 학습

다음은 [2입력 1출력] 인공 신경의 신경망 학습을 나타내는 수식입니다.

```
w1 -= lr*w1E
w2 -= lr*w2E
 b -= lr*bE
```

lr은 learning rate의 약자로 학습률을 의미합니다. w1E, w2E는 가중치 역전파 오차, bE는 편향 역전파 오차로 딥러닝 제 6 공식에서 구한 값입니다.

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
331_5.ino

```
double x1 = 2, x2 = 3;
double w1 = 3, w2 = 4;
double b = 1;
double yT = 27;
double lr  = 0.01;

void setup() {

        Serial.begin(115200);

        double y = x1*w1 + x2*w2 + 1*b;
        double E = (y - yT)*(y - yT) / 2;
        double yE = y - yT;
        double w1E = yE*x1;
        double w2E = yE*x2;
        double bE = yE*1;
        w1 -= lr*w1E;
        w2 -= lr*w2E;
        b -= lr*bE;
        Serial.print("w1 : "); Serial.println(w1, 6);
        Serial.print("w2 : "); Serial.println(w2, 6);
        Serial.print("b : "); Serial.println(b, 6);

}

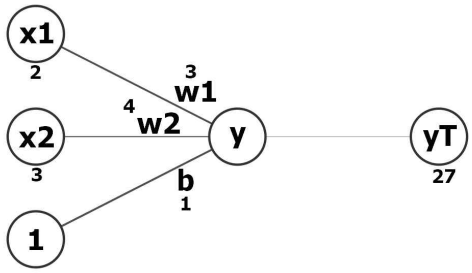
void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
w1 : 3.160000
w2 : 4.240000
b : 1.080000
```

딥러닝 반복 학습해 보기

지금까지의 과정을 반복해서 학습해 봅니다. 다음 그림을 살펴봅니다.



이 그림에서 입력값 x1, x2는 각각 2, 3, 가중치 w1, w2는 각각 3, 4, 편향 b는 1이고 목표값 yT는 27 입니다. x1, x2, yT를 이용하여 w1, w2, b에 대해 학습을 수행해 봅니다.

*** 이 값들은 임의의 값들입니다. 다른 값들을 사용하여 학습을 수행할 수도 있습니다.

1. 다음과 같이 이전 예제를 수정합니다.
331_6.ino

```
double x1 = 2, x2 = 3;
double w1 = 3, w2 = 4;
double b = 1;
double yT = 27;
double lr  = 0.01;

void setup() {

        Serial.begin(115200);

        for(int epoch=0;epoch<200;epoch++) {

                double y = x1*w1 + x2*w2 + 1*b;
                double E = (y - yT)*(y - yT) / 2;
                double yE = y - yT;
                double w1E = yE*x1;
                double w2E = yE*x2;
                double bE = yE*1;
                w1 -= lr*w1E;
                w2 -= lr*w2E;
                b -= lr*bE;

                Serial.print("epoch = "); Serial.println(epoch);
                Serial.print("y : "); Serial.println(y, 3);
                Serial.print("w1 : "); Serial.println(w1, 3);
```

```
                    Serial.print("w2 : "); Serial.println(w2, 3);
                    Serial.print("b : "); Serial.println(b, 3);

                    if(E< 0.0000001) break;

        }

}

void loop() {
}
```

2.  업로드를 수행합니다. 다음은 실행 결과 화면입니다.

```
epoch = 64
y : 26.999
w1 : 4.143
w2 : 5.714
b : 1.571
epoch = 65
y : 27.000
w1 : 4.143
w2 : 5.714
b : 1.571
```

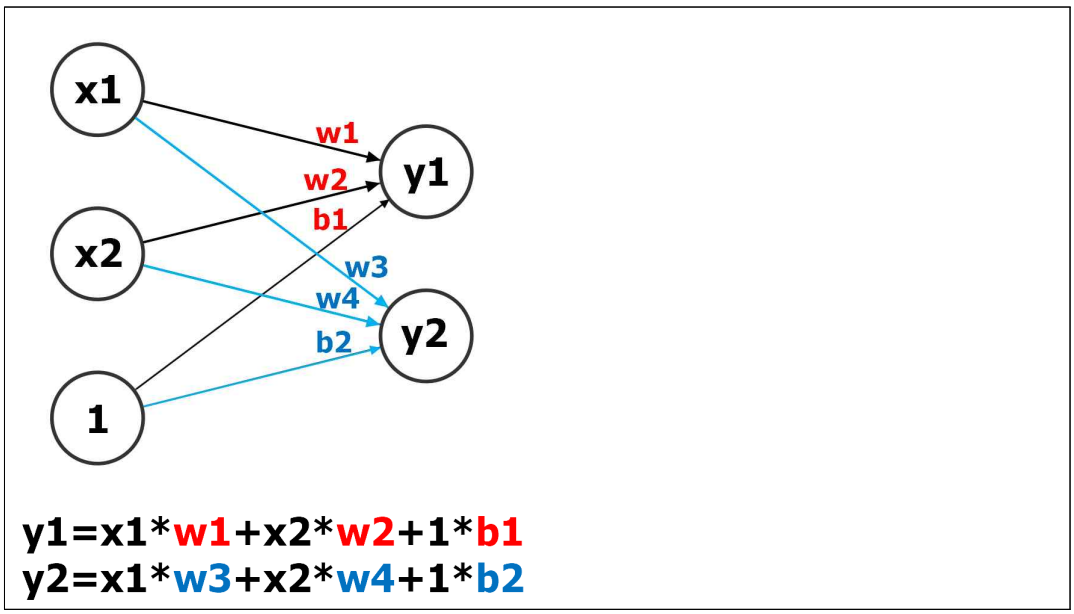
epoch 값이 65일 때 학습이 완료되는 것을 볼 수 있습니다. 가중치 w1, w2는 각각 4.143, 5.714, 편향 b는 1.571에 수렴합니다.

02 2입력 2출력 인공 신경망

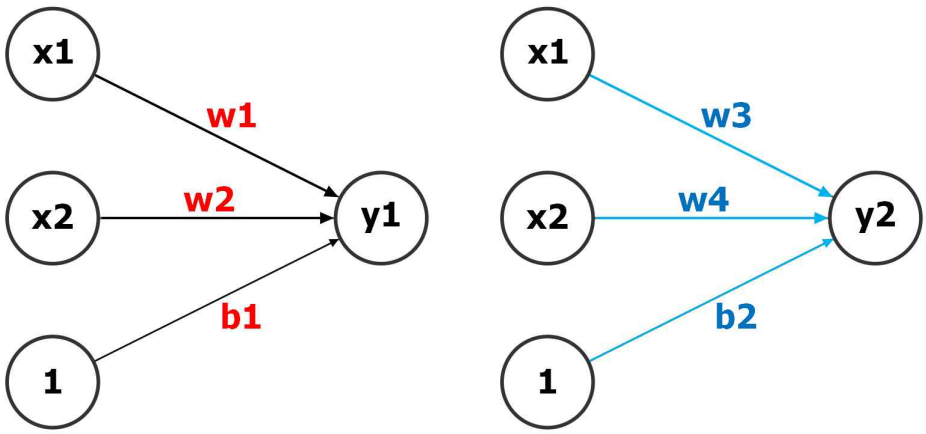
다음은 [2입력 2출력] 인공 신경망에 대해 딥러닝 7 공식을 확장하고 구현해 봅니다. 참고로 [2입력 2출력] 인공 신경망에 대해 적용되는 딥러닝 7 공식은 [m입력 n출력, m, n은 자연수]의 일반적인 인공 신경망에도 적용됩니다.

딥러닝 제 1 공식 : 순전파

다음은 [2입력 2출력] 인공 신경망의 딥러닝 제 1 공식을 나타냅니다.



위의 수식은 순전파 수식입니다. y1, y2는 예측값, x1, x2는 입력값, w1, w2, w3, w4는 가중치, b1, b2는 편향입니다. [2입력 2출력] 인공 신경망은 다음 그림과 같이 [2입력 1출력] 인공 신경 2개로 구성됩니다.



다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.

332.ino

```
double x1 = 2, x2 = 3;
double w1 = 3, w2 = 4;
double w3 = 5, w4 = 6;
double b1 = 1, b2 = 2;

void setup() {

        Serial.begin(115200);

        double y1 = x1*w1 + x2*w2 + 1*b1;
        double y2 = x1*w3 + x2*w4 + 1*b2;
        Serial.print("y1 : "); Serial.print(y1, 6);
        Serial.print(", y2 : "); Serial.println(y2, 6);

}

void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
y1 : 19.000000, y2 : 30.000000
```

딥러닝 제 2 공식 : 평균 제곱 오차

다음은 [2입력 2출력] 인공 신경망의 딥러닝 제 2 공식을 나타냅니다.

```
E = (y1-y1T)*(y1-y1T)/2
  + (y2-y2T)*(y2-y2T)/2
```

[2입력 2출력] 인공 신경망은 [2입력 1출력] 인공 신경 2개로 구성됩니다. 따라서 각 인공 신경 오차가 더해집니다.

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
332_2.ino

```
double x1 = 2, x2 = 3;
double w1 = 3, w2 = 4;
double w3 = 5, w4 = 6;
double b1 = 1, b2 = 2;
double y1T = 27, y2T = -30;

void setup() {

        Serial.begin(115200);

        double y1 = x1*w1 + x2*w2 + 1*b1;
        double y2 = x1*w3 + x2*w4 + 1*b2;
        double E = (y1-y1T)*(y1-y1T)/2+(y2-y2T)*(y2-y2T)/2;
        Serial.print("E : "); Serial.println(E, 6);

}

void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
E : 1832.000000
```

딥러닝 제 3 공식 : 역전파 오차

다음은 [2입력 2출력] 인공 신경의 딥러닝 제 3 공식을 나타냅니다.

```
y1E=y1-y1T
y2E=y2-y2T
```

[2입력 2출력] 인공 신경망은 [2입력 1출력] 인공 신경 2개로 구성됩니다. 따라서 각 출력에 대해 역전파 오차가 계산됩니다.

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
332_3.ino

```
double x1 = 2, x2 = 3;
double w1 = 3, w2 = 4;
double w3 = 5, w4 = 6;
double b1 = 1, b2 = 2;
double y1T = 27, y2T = -30;

void setup() {

        Serial.begin(115200);

        double y1 = x1*w1 + x2*w2 + 1*b1;
        double y2 = x1*w3 + x2*w4 + 1*b2;
        double E = (y1-y1T)*(y1-y1T)/2+(y2-y2T)*(y2-y2T)/2;
        double y1E = y1 - y1T;
        double y2E = y2 - y2T;
        Serial.print("y1E : "); Serial.print(y1E, 6);
        Serial.print(", y2E : "); Serial.println(y2E, 6);

}

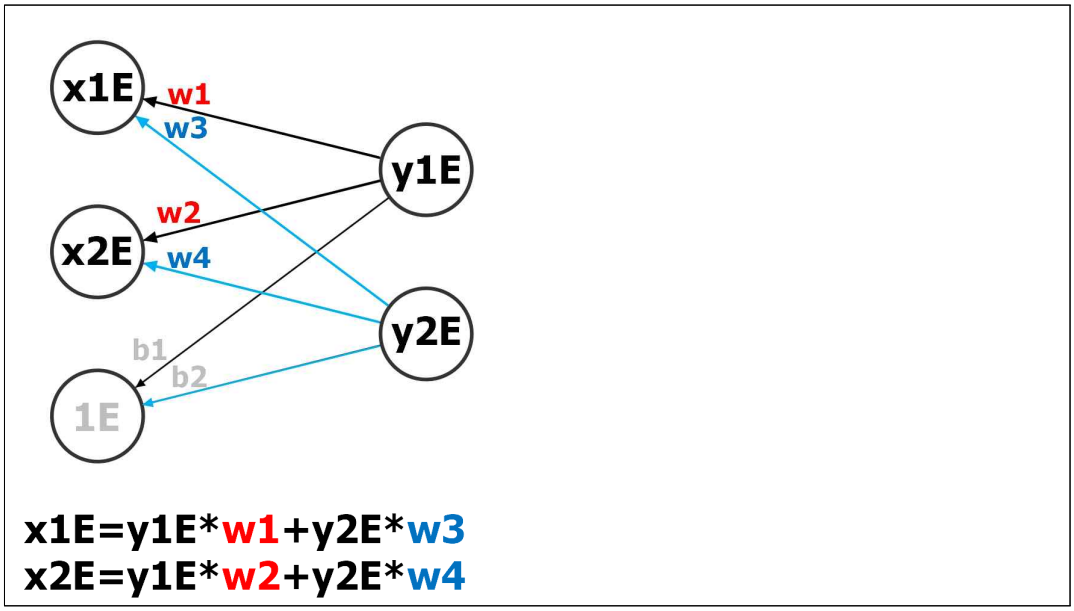
void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
y1E : -8.000000, y2E : 60.000000
```

딥러닝 제 4 공식 : 입력 역전파

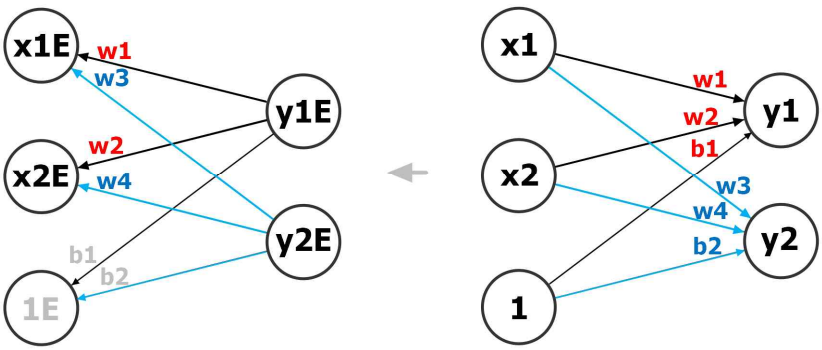
다음은 [2입력 2출력] 인공 신경망의 딥러닝 제 4 공식을 나타냅니다.



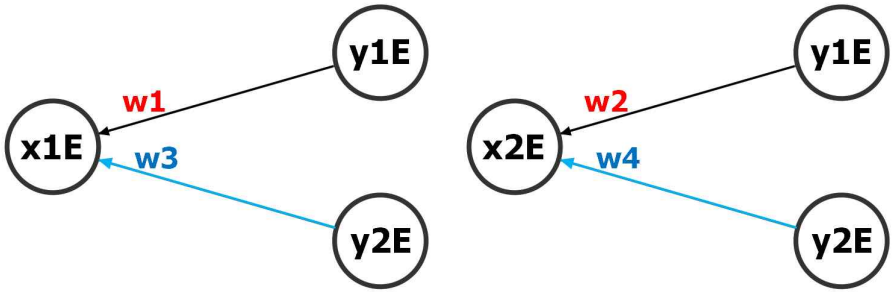
x1E, x2E는 입력 역전파, y1E, y2E는 역전파 오차로 딥러닝 제 3 공식에서 구한 값입니다. 회색으로 표시된 1E는 숫자 1의 오차라는 의미로 사용하지 않는 부분입니다. 딥러닝 제 4 공식은 다음과 같은 순서로 유도할 수 있습니다.

1. 딥러닝 제 1 공식의 그림을 복사합니다.
2. y1 -> y1E, y2 -> y2E, x1 -> x1E, x2 -> x2E로 변경합니다.
3. 화살표 방향을 반대로 합니다.
4. 1E, b1, b2는 사용하지 않습니다.

다음 그림을 참조합니다.



[2입력 2출력] 입력 역전파 그림은 다음과 같이 2개 오차 역전파 망으로 나누어 생각할 수 있습니다.



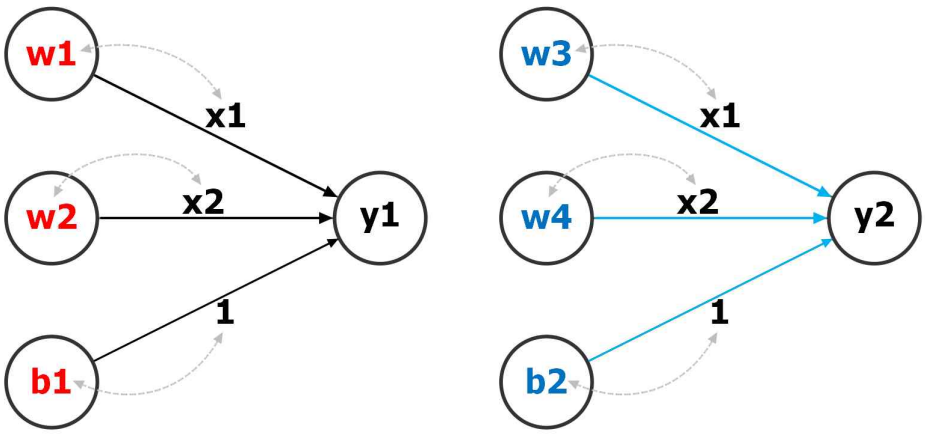
y1E, y2E는 w1, w3를 따라 x1E로 전파됩니다. 또, y1E, y2E는 w2, w4를 따라 x2E로 전파됩니다.

딥러닝 제 5 공식 : 가중치, 편향 순전파

다음은 [2입력 2출력] 인공 신경망의 가중치와 편향의 순전파를 나타내는 식과 그림입니다.

y1=x1*w1+x2*w2+1*b1
y2=x1*w3+x2*w4+1*b2 (딥러닝 제 1 공식)

y1=w1*x1+w2*x2+b1*1
y2=w3*x1+w4*x2+b2*1 (딥러닝 제 5 공식)



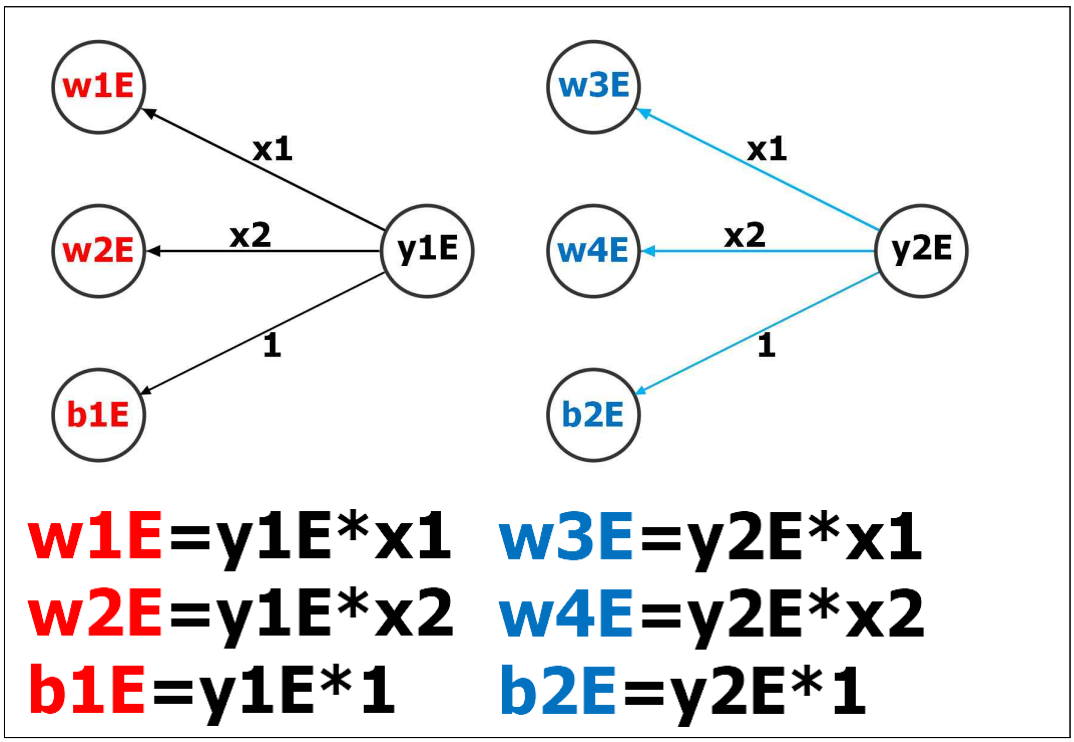
딥러닝 제 5 공식은 다음과 같은 순서로 유도할 수 있습니다.

1. 딥러닝 제 1 공식을 복사합니다.
2. x1과 w1, x2와 w2, 1과 b1,
   x1과 w3, x2와 w4, 1과 b2를 교환하여 딥러닝 제 5 공식을 유도합니다.
3. 딥러닝 제 1 공식의 그림과 같은 형태로 그림을 그립니다.

딥러닝 제 5 공식은 다음에 나올 딥러닝 제 6 공식을 유도하기 위해 사용하며, 실제로 딥러닝 제 1 공식과 같습니다. 딥러닝 제 5 공식은 구현을 위해 사용하지는 않습니다.

딥러닝 제 6 공식 : 가중치와 편향 역전파

다음은 [2입력 2출력] 인공 신경망의 가중치와 편향의 역전파를 나타내는 그림과 수식입니다.



w1E, w2E, w3E, w4E는 가중치 역전파 오차, b1E, b2E는 편향 역전파 오차를 의미합니다. y1E, y2E는 역전파 오차로 딥러닝 제 3 공식에서 구한 값입니다. 딥러닝 제 6 공식은 다음과 같은 순서로 유도할 수 있습니다.

1. 딥러닝 제 5 공식의 그림을 복사합니다.
2. y1 -> y1E, w1 -> w1E, w2 -> w2E, b1 -> b1E,
   y2 -> y2E, w3 -> w3E, w4 -> w4E, b2 -> b2E로 변경합니다.
3. 화살표 방향을 반대로 합니다.

딥러닝 제 6 공식은 실제로 편미분을 연쇄적으로 적용하여 얻은 공식입니다. 편미분을 이용하여 역전파를 유도하는 방법은 부록을 참조합니다.

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
332_4.ino

```
double x1 = 2, x2 = 3;
double w1 = 3, w2 = 4;
double w3 = 5, w4 = 6;
double b1 = 1, b2 = 2;
double y1T = 27, y2T = -30;

void setup() {

        Serial.begin(115200);

        double y1 = x1*w1 + x2*w2 + 1*b1;
        double y2 = x1*w3 + x2*w4 + 1*b2;
        double E = (y1-y1T)*(y1-y1T)/2+(y2-y2T)*(y2-y2T)/2;
        double y1E = y1 - y1T;
        double y2E = y2 - y2T;
        double w1E = y1E*x1;
        double w2E = y1E*x2;
        double b1E = y1E*1;
        double w3E = y2E*x1;
        double w4E = y2E*x2;
        double b2E = y2E*1;
        Serial.print("w1E : "); Serial.print(w1E, 6);
        Serial.print(", w2E : "); Serial.print(w2E, 6);
        Serial.print(", b1E : "); Serial.println(b1E, 6);
        Serial.print("w3E : "); Serial.print(w3E, 6);
        Serial.print(", w4E : "); Serial.print(w4E, 6);
        Serial.print(", b2E : "); Serial.println(b2E, 6);

}

void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
w1E : -16.000000, w2E : -24.000000, b1E : -8.000000
w3E : 120.000000, w4E : 180.000000, b2E : 60.000000
```

07 딥러닝 제 7 공식 : 신경망 학습

다음은 [2입력 2출력] 인공 신경망의 신경망 학습을 나타내는 수식입니다.

```
w1 -= lr*w1E
w2 -= lr*w2E
 b1 -= lr*b1E
w3 -= lr*w3E
w4 -= lr*w4E
 b2 -= lr*b2E
```

lr은 learning rate의 약자로 학습률을 의미합니다. w1E, w2E, w3E, w4E는 가중치 역전파 오차, b1E, b2E는 편향 역전파 오차로 딥러닝 제 6 공식에서 구한 값입니다.

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
332_5.ino

```
double x1 = 2, x2 = 3;
double w1 = 3, w2 = 4;
double w3 = 5, w4 = 6;
double b1 = 1, b2 = 2;
double y1T = 27, y2T = -30;
double lr  = 0.01;

void setup() {

        Serial.begin(115200);

        double y1 = x1*w1 + x2*w2 + 1*b1;
        double y2 = x1*w3 + x2*w4 + 1*b2;
        double E = (y1-y1T)*(y1-y1T)/2+(y2-y2T)*(y2-y2T)/2;
        double y1E = y1 - y1T;
        double y2E = y2 - y2T;
        double w1E = y1E*x1;
        double w2E = y1E*x2;
        double b1E = y1E*1;
        double w3E = y2E*x1;
        double w4E = y2E*x2;
        double b2E = y2E*1;
        w1 -= lr*w1E;
        w2 -= lr*w2E;
        b1 -= lr*b1E;
        w3 -= lr*w3E;
        w4 -= lr*w4E;
        b2 -= lr*b2E;
        Serial.print("w1 : "); Serial.print(w1, 6);
        Serial.print(", w2 : "); Serial.print(w2, 6);
        Serial.print(", b1 : "); Serial.println(b1, 6);
        Serial.print("w3 : "); Serial.print(w3, 6);
        Serial.print(", w4 : "); Serial.print(w4, 6);
        Serial.print(", b2 : "); Serial.println(b2, 6);

}

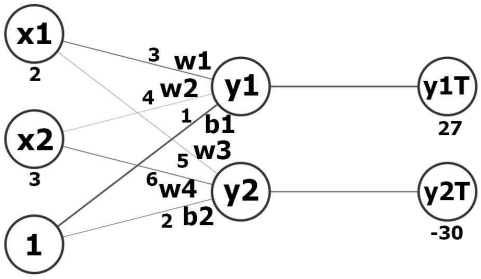
void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
w1 : 3.160000, w2 : 4.240000, b1 : 1.080000
w3 : 3.800000, w4 : 4.200000, b2 : 1.400000
```

딥러닝 반복 학습해 보기

지금까지의 과정을 반복해서 학습해 봅니다. 다음 그림을 살펴봅니다.



이 그림에서 입력값 x1, x2는 각각 2, 3, 가중치 w1, w2, 편향 b1은 각각 3, 4, 1, 가중치 w3, w4, 편향 b2는 각각 5, 6, 2이고 목표값 y1T, y2T는 각각 27, -30입니다. x1, x2, y1T, y2T 를 이용하여 w1, w2, b1, w3, w4, b2에 대해 학습을 수행해 봅니다.

*** 이 값들은 임의의 값들입니다. 다른 값들을 사용하여 학습을 수행할 수도 있습니다.

1. 다음과 같이 이전 예제를 수정합니다.
332_6.ino

```
double x1 = 2, x2 = 3;
double w1 = 3, w2 = 4;
double w3 = 5, w4 = 6;
double b1 = 1, b2 = 2;
double y1T = 27, y2T = -30;
double lr  = 0.01;

void setup() {

        Serial.begin(115200);

        for(int epoch=0;epoch<200;epoch++) {

                double y1 = x1*w1 + x2*w2 + 1*b1;
                double y2 = x1*w3 + x2*w4 + 1*b2;
                double E = (y1-y1T)*(y1-y1T)/2 + (y2-y2T)*(y2-y2T)/2;
                double y1E = y1 - y1T;
                double y2E = y2 - y2T;
                double w1E = y1E*x1;
                double w2E = y1E*x2;
                double b1E = y1E*1;
                double w3E = y2E*x1;
                double w4E = y2E*x2;
                double b2E = y2E*1;
                w1 -= lr*w1E;
```

```
                    w2 -= lr*w2E;
                    b1 -= lr*b1E;
                    w3 -= lr*w3E;
                    w4 -= lr*w4E;
                    b2 -= lr*b2E;

                    Serial.print("epoch = "); Serial.println(epoch);
                    Serial.print(" y1 : "); Serial.println(y1, 3);
                    Serial.print(" y2 : "); Serial.println(y2, 3);
                    Serial.print(" w1 : "); Serial.println(w1, 3);
                    Serial.print(" w2 : "); Serial.println(w2, 3);
                    Serial.print(" b1 : "); Serial.println(b1, 3);
                    Serial.print(" w3 : "); Serial.println(w3, 3);
                    Serial.print(" w4 : "); Serial.println(w4, 3);
                    Serial.print(" b2 : "); Serial.println(b2, 3);

                    if(E< 0.0000001) break;

            }

}

void loop() {
}
```

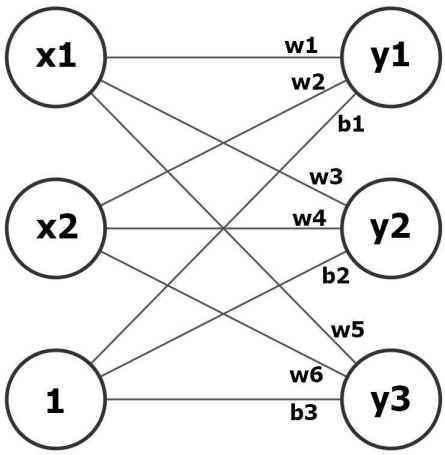
2.  업로드를 수행합니다. 다음은 실행 결과 화면입니다.

```
epoch = 79
 y1 : 27.000
 y2 : -30.000
 w1 : 4.143
 w2 : 5.714
 b1 : 1.571
 w3 : -3.571
 w4 : -6.857
 b2 : -2.286
```

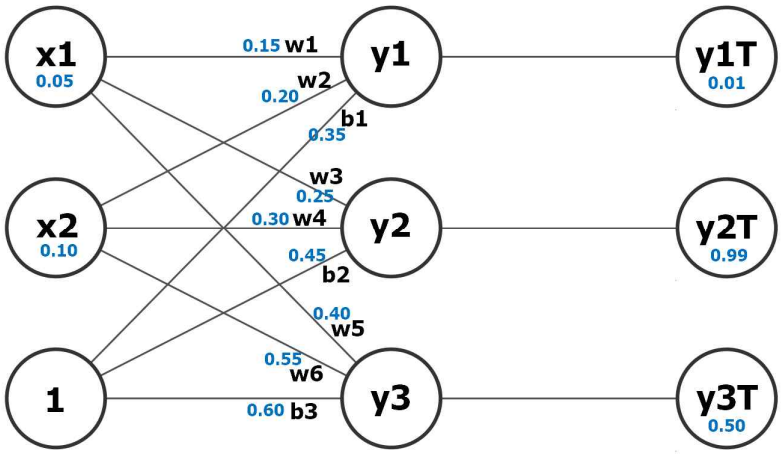
epoch 값이 79일 때 학습이 완료되는 것을 볼 수 있습니다. 가중치 w1, w2는 각각 4.143, 5.714, 편향 b1은 1.571, 가중치 w3, w4는 각각 -3.571, -6.857 편향 b2는 -2.286에 수렴합니다.

연습문제 1

1. 다음은 [2입력 3출력]의 인공 신경망입니다. 이 인공 신경망의 딥러닝 7 공식을 구합니다.

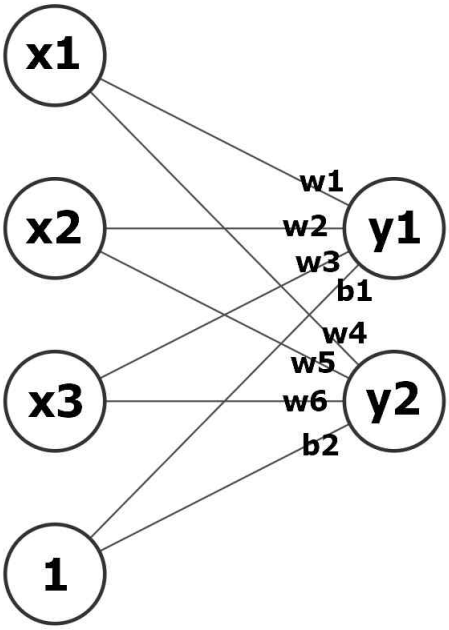


2. [문제 1]에서 구한 딥러닝 7 공식을 이용하여 다음과 같이 초기화된 인공 신경망을 구현하고 학습시켜 봅니다.

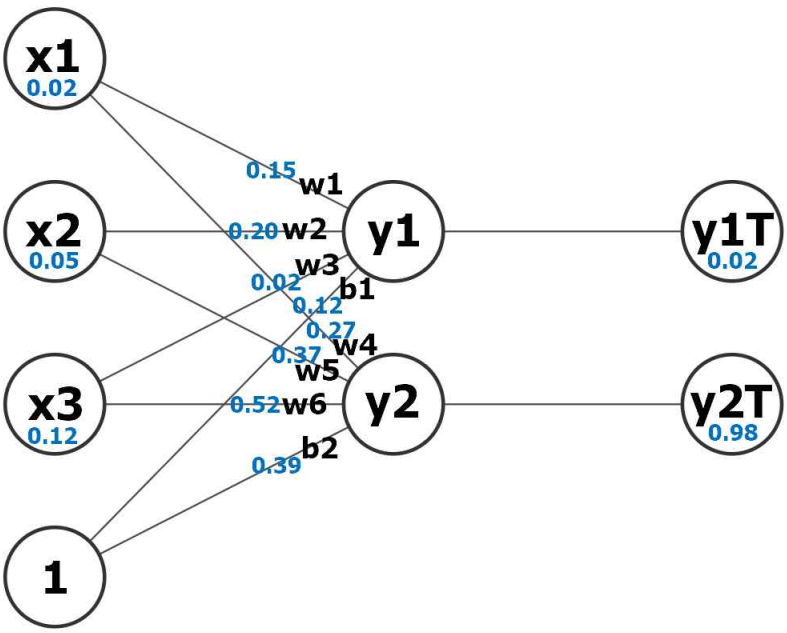


연습문제 2

1. 다음은 [3입력 2출력]의 인공 신경망입니다. 이 인공 신경망의 딥러닝 7 공식을 구합니다.



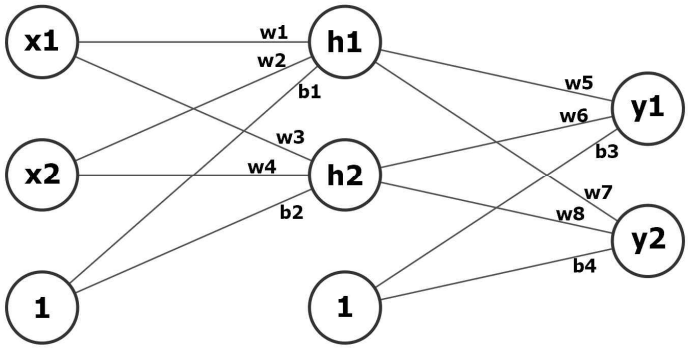
2. [문제 1]에서 구한 딥러닝 7 공식을 이용하여 다음과 같이 초기화된 인공 신경망을 구현하고 학습시켜 봅니다.



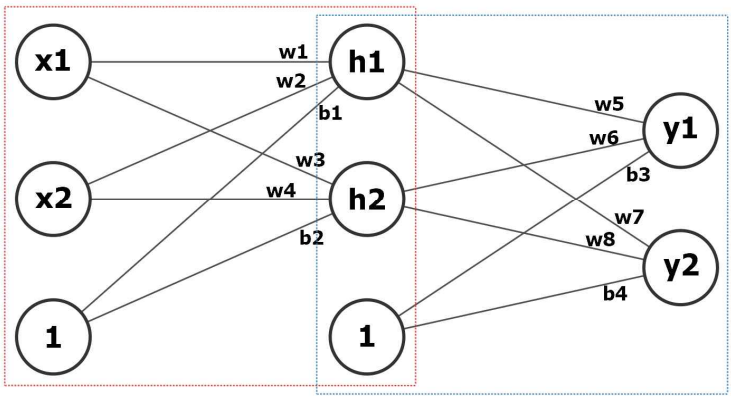
03 2입력 2은닉 2출력 인공 신경망

여기서는 앞에서 정리한 딥러닝 7 공식을 이용하여 2입력 2은닉 2출력 인공 신경망을 학습시켜 봅니다.

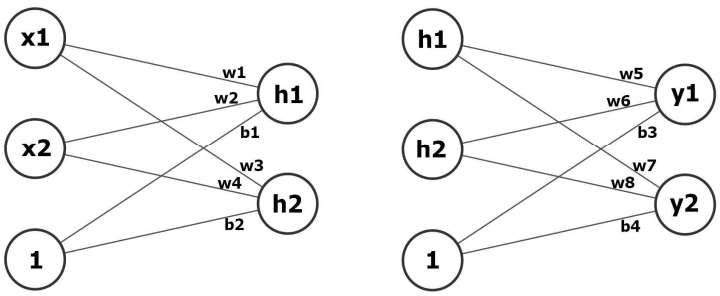
다음 그림은 [2입력 2은닉 2출력]으로 구성된 인공 신경망을 나타냅니다. 인공 신경망에서 입력층과 출력층 사이에 오는 층을 은닉층이라고 합니다.



[2입력 2은닉 2출력] 인공 신경망은 다음 그림과 같이 2개의 [2입력 2출력] 인공 신경으로 구성됩니다.



그래서 다음과 같이 [2입력 2출력] 인공 신경 2개로 나눌 수 있습니다.



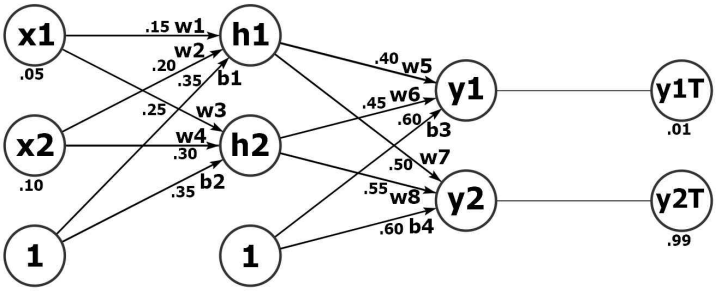
인공 신경망 수식 정리하기

2입력 2은닉 2출력 인공 신경망의 수식은 다음과 같습니다.

딥러닝 제 1 공식 : 순전파	h1=x1*w1+x2*w2+b1 h2=x1*w3+x2*w4+b2 y1=h1*w5+h2*w6+b3 y2=h1*w7+h2*w8+b4
딥러닝 제 2 공식 : 오차	E=(y1-y1T)^2/2+(y2-y2T)^2/2
딥러닝 제 3 공식 : 역전파 오차	y1E=y1-y1T y2E=y2-y2T
딥러닝 제 4 공식 : 입력 역전파	h1E=y1E*w5+y2E*w7 h2E=y1E*w6+y2E*w8
딥러닝 제 6 공식 : 가중치, 편향 역전파	w5E=y1E*h1 w6E=y1E*h2 w7E=y2E*h1 w8E=y2E*h2 b3E=y1E*1 b4E=y2E*1 w1E=h1E*x1 w2E=h1E*x2 w3E=h2E*x1 w4E=h2E*x2 b1E=h1E*1 b2E=h2E*1

인공 신경망 구현하기

지금까지 정리한 수식으로 신경망을 학습시켜 봅니다. 다음 그림을 살펴봅니다.



*** 이 값들은 임의의 값들입니다. 다른 값들을 사용하여 학습을 수행할 수도 있습니다.

1. 다음과 같이 예제를 작성합니다.
333.ino

```
double x1 = 0.05, x2 = 0.10;

double w1 = 0.15, w2 = 0.20;
double w3 = 0.25, w4 = 0.30;
double b1 = 0.35, b2 = 0.35;

double w5 = 0.40, w6 = 0.45;
double w7 = 0.50, w8 = 0.55;
double b3 = 0.60, b4 = 0.60;

double y1T = 0.01, y2T = 0.99;

double lr  = 0.01;

void setup() {

        Serial.begin(115200);

        for(int epoch=0;epoch<2000;epoch++) {

                double h1 = x1*w1 + x2*w2 + 1*b1;
                double h2 = x1*w3 + x2*w4 + 1*b2;

                double y1 = h1*w5 + h2*w6 + 1*b3;
                double y2 = h1*w7 + h2*w8 + 1*b4;
```

```
double E = ((y1-y1T)*(y1-y1T) + (y2-y2T)*(y2-y2T))/2;

double y1E = y1 - y1T;
double y2E = y2 - y2T;

double w5E = y1E*h1;
double w6E = y1E*h2;
double w7E = y2E*h1;
double w8E = y2E*h2;
double b3E = y1E*1;
double b4E = y2E*1;

double h1E = y1E*w5 + y2E*w7;
double h2E = y1E*w6 + y2E*w8;

double w1E = h1E*x1;
double w2E = h1E*x2;
double w3E = h2E*x1;
double w4E = h2E*x2;
double b1E = h1E*1;
double b2E = h2E*1;

w5 -= lr*w5E;
w6 -= lr*w6E;
w7 -= lr*w7E;
w8 -= lr*w8E;
b3 -= lr*b3E;
b4 -= lr*b4E;

w1 -= lr*w1E;
w2 -= lr*w2E;
w3 -= lr*w3E;
w4 -= lr*w4E;
b1 -= lr*b1E;
b2 -= lr*b2E;

Serial.print("epoch = "); Serial.println(epoch);
Serial.print(" y1 : "); Serial.println(y1, 3);
Serial.print(" y2 : "); Serial.println(y2, 3);
```

```
                if(E<0.0000001) break;

        }

}

void loop() {
}
```

2.  업로드를 수행합니다. 다음은 실행 결과 화면입니다.

```
epoch = 665
 y1 : 0.010
 y2 : 0.990
```

학습 회수에 따라 y1, y2값이 바뀌는 것을 확인합니다. y1, y2값이 각각 0.01, 0.99에 가까워지는 것을 확인합니다. 입력값 0.05, 0.10에 대해 목표값은 0.01, 0.99입니다.

04 딥러닝 7 공식 구현하기 : PyTorch

여기서는 딥러닝 7 공식을 PyTorch로 구현해 보며 PyTorch의 내부적인 동작을 이해해 봅니다. PyTorch의 내부 동작을 잘 이해하여 PyTorch의 활용도를 높일 수 있도록 합니다.



01 PyTorch 실습 환경 구성하기

먼저 PyTorch 실습 환경을 구성합니다. 이 책에서는 Thonny IDE 상에서 실습을 진행합니다. Thonny IDE는 간결한 실습 환경을 제공하여 초보자가 사용하기에 적합합니다.

Thonny IDE 설치하기

1. 다음과 같이 검색합니다.



2. 다음 사이트로 들어갑니다.



3. 다음 페이지가 열립니다. 화면 우측 상단에 있는 ❶ [Windows] 링크를 누릅니다. ❷ 가장 위쪽에 있는 프로그램을 마우스 클릭합니다.



4. 다음과 같이 설치 프로그램이 다운로드 됩니다. 프로그램을 실행시켜 설치를 진행합니다.



파이썬 실습 환경 설정하기

여기서는 파이썬 실습 환경을 설정해 봅니다.

1. 데스크 탑 좌측 하단에 있는 [검색] 창을 찾아 [thonny]를 입력합니다.



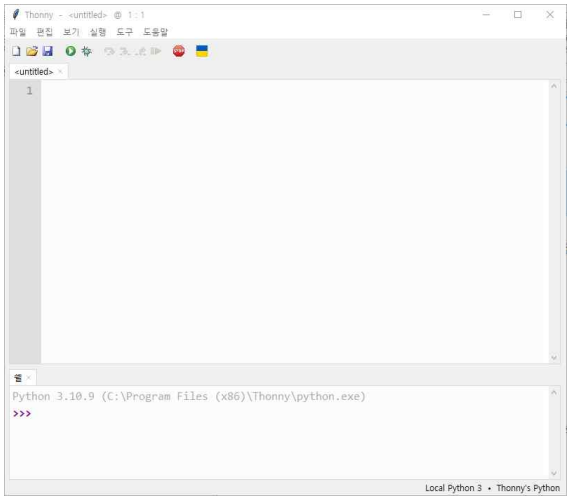
2. 다음 프로그램을 실행합니다.



3. 처음엔 다음과 같이 [언어 선택과 초기 설정] 창이 뜹니다. 언어를 [한국어]로 선택한 후, [Let's go!] 버튼을 누릅니다.



4. 다음과 같이 [Thonny IDE] 프로그램이 실행됩니다. 하단에는 Thonny IDE가 기본적으로 지원하는 PC용 파이썬 쉘이 실행됩니다.

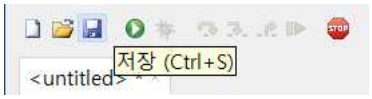


파이썬 프로그램 작성하기

1. 다음과 같이 <untitled> 파일 상에 프로그램을 작성합니다.



2. 다음과 같이 [저장] 버튼을 누릅니다.

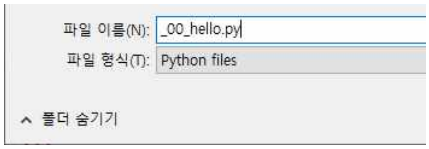


3. 다음과 같이 [다른 이름으로 저장] 창이 뜹니다. ❶ [바탕 화면]으로 이동한 후, ❷ [새 폴더]를 생성한 후, ❸ [pyLabs] 디렉터리를 생성합니다.



4. [pyLabs] 디릭터리로 이동한 후, [_00_hello.py]로 저장합니다.





5. 다음과 같이 파일이 저장됩니다.



파이썬 프로그램 실행하기

1. 다음과 같이 프로그램을 실행시킵니다. 초록색 삼각형 아이콘을 마우스로 눌러줍니다.

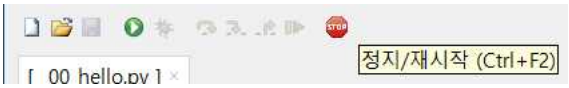


2. 다음은 실행 결과 화면입니다.



파이썬 프로그램 종료하기

1. 다음과 같이 프로그램을 종료합니다. 빨간색 [STOP] 아이콘을 마우스로 눌러줍니다.



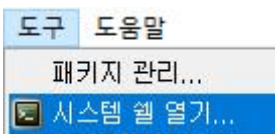
2. 다음은 프로그램 종료 화면입니다.



PyTorch 라이브러리 설치하기

다음은 PyTorch 라이브러리를 설치합니다.

1. [도구]--[시스템 쉘 열기...] 메뉴를 선택합니다.

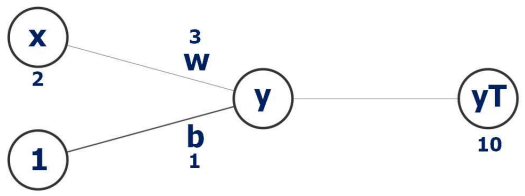


2. 다음과 같이 명령을 수행합니다. torch, numpy 라이브러리를 설치합니다.

```
C:₩Users₩edu₩Desktop₩pyLabs>pip3 install torch
C:₩Users₩edu₩Desktop₩pyLabs>pip3 install numpy
C:₩Users₩edu₩Desktop₩pyLabs>pip3 install matplotlib
```

02 1입력 1출력 인공 신경 구현하기

다음은 [입력1 출력1]의 인공 신경 학습에 사용할 신경망을 나타냅니다.



$$X = [2]$$
$$T = [10]$$
$$W = [3]$$
$$B = [1]$$

X를 입력값으로, T를 목표값으로 하여 가중치 W와 편향 B에 대해 PyTorch로 학습해 봅니다.

1. 다음과 같이 예제를 작성합니다.

342.py

```
01 import torch
02 import torch.nn as nn
03 import torch.optim as optim
04
05 X = torch.FloatTensor([[2]]) # 입력데이터
06 YT = torch.FloatTensor([[10]]) # 목표데이터(라벨)
07 W = torch.FloatTensor([[3]]) # 가중치
08 B = torch.FloatTensor([1]) # 편향
09
10 model = nn.Sequential(
11       nn.Linear(1,1) # 입력 층, 출력 층 노드의 개수를 각각 1로 설정합니다.
12 ) # 신경망 모양 결정(W, B 내부적 준비)
13 print(model) # 신경망 모양 출력
14
15 with torch.no_grad():
16       model[0].weight = nn.Parameter(W)
17       model[0].bias = nn.Parameter(B)
18
19 loss_fn = nn.MSELoss() # 2공식, 오차 계산 함수
20 optimizer = optim.SGD(model.parameters(), lr=0.01)
21
22 for epoch in range(1000):
23
```

```
24        Y = model(X) # 1공식, 순전파
25        E = loss_fn(Y, YT) # 2공식, 오차계산
26        optimizer.zero_grad()
27        E.backward() # 6공식, 오차역전파
28        optimizer.step() # 7공식, 학습
29
30        if epoch%100 == 99:
31                print(epoch, E.item())
32
33 print(f'W = {model[0].weight.data}')
34 print(f'B = {model[0].bias.data}')
35
36 Y = model(X)
37 print(Y.data)
```

01 : import문을 이용하여 torch 모듈을 불러옵니다. torch 모듈은 PyTorch에서 사용되는 텐서 라이브러리를 말하여 Facebook에서 개발한 오픈소스 머신러닝 라이브러리입니다.
02 : import문을 이용하여 torch.nn 모듈을 nn이라는 이름으로 불러옵니다. nn 모듈은 인공 신경망 구성에 사용하는 라이브러리입니다. nn 모듈에는 완전 연결층에 사용하는 nn.Linear, 합성곱층에 사용하는 nn.Conv2d, RNN, LSTM과 같은 Layer들, ReLU, Sigmoid와 같은 활성화 함수, 오차 계산에 사용하는 MSELoss, CrossEntropyLoss 함수 등이 있습니다.
03 : import문을 이용하여 torch.optim 모듈을 optim이라는 이름으로 불러옵니다. optim 모듈은 인공 신경망 학습에 사용하는 최적화 함수를 포함한 라이브러리입니다. 예를 들어, SGD, Adam과 같은 최적화 함수들이 있습니다.
05~07 : X, YT, W 변수를 이차 배열의 torch.FloatTensor로 초기화합니다.
08 : B 변수를 일차 배열의 torch.FloatTensor로 초기화합니다.
10~12 : nn.Sequential 클래스를 이용하여 인공 신경망을 생성합니다.
11 : nn.Linear 클래스를 이용하여 신경망 층을 생성합니다. 입력 노드 1개, 출력 노드 1개로 구성된 인공 신경망 층을 생성합니다.
13 : 모델의 모양을 출력해 줍니다.
15~17 : 인공 신경망 0층에 임의로 설정된 가중치와 편향을 W, B로 변경합니다. torch.no_grad 객체는 gradient 연산 비활성화 객체로 15~16줄 수행 시 역전파 계산을 비활성화합니다.
19 : 오차 계산 함수로 nn.MSErrorLoss 함수를 사용합니다.
20 : 최적화 함수는 확률적 경사 하강(sgd : stochastic gradient descent) 함수인 optim.SGD를 사용합니다. 학습할 가중치와 매개 변수를 model.parameters() 함수로 넘겨줍니다. 학습률은 0.01로 설정합니다.
22~31 : 인공 신경망에 대한 학습을 합니다. 여기서는 1000회 학습을 수행하도록 합니다.
24 : 순전파를 합니다. 딥러닝 제 1 공식을 수행하는 부분입니다.
25 : 오차 계산을 합니다. 딥러닝 제 2 공식을 수행하는 부분입니다.
26 : optimizer.zero_grad 함수를 호출하여 오차 역전파 변수를 초기화합니다.
27 : 오차 역전파를 합니다. 딥러닝 제 6 공식을 수행하는 부분입니다.

28 : W,B에 대한 학습을 수행합니다. 딥러닝 제 7 공식을 수행하는 부분입니다.
30~31 : 100번마다 오차 함수 계산 결과를 출력합니다.
33~34 : 학습이 끝난 W, B 값을 출력해 봅니다.
36~37 : 학습이 끝난 신경망을 이용하여 예측을 수행해 봅니다.

2.  예제를 실행합니다. 다음은 실행 결과 화면입니다.

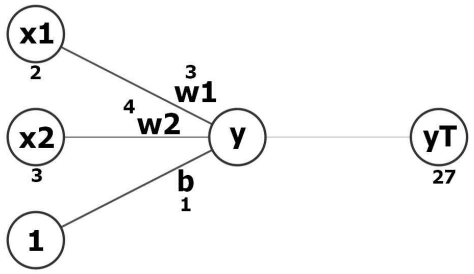
```
Sequential(
  (0): Linear(in_features=1, out_features=1, bias=True)
)
```

```
999 8.185452315956354e-12
W= tensor([[4.2000]])
B= tensor([1.6000])
tensor([[10.0000]])
```

신경망의 구조와 학습 결과를 볼 수 있습니다.

03 2입력 1출력 인공 신경 구현하기

다음은 입력2 출력1의 인공 신경 학습에 사용할 신경망을 나타냅니다.



$$X = [2 \quad 3]$$
$$T = [27]$$
$$W = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$
$$B = [1]$$

X를 입력 값으로, T를 목표 값으로 하여 가중치 W와 편향 B에 대해 PyTorch로 학습해 봅니다.

1. 다음과 같이 예제를 수정합니다.

343.py

```
01~04 # 이전 예제와 같습니다.
05 X = torch.FloatTensor([[2,3]]) # 입력데이터
06 YT = torch.FloatTensor([[27]]) # 목표데이터(라벨)
07 W = torch.FloatTensor([[3,4]]) # 가중치
08 B = torch.FloatTensor([1]) # 편향
09
10 model = nn.Sequential(
11       nn.Linear(2,1) # 입력 층, 출력 층 노드의 개수를 각각 1로 설정합니다.
12 ) # 신경망 모양 결정(W, B 내부적 준비)
13 print(model) # 신경망 모양 출력
14~끝 # 이전 예제와 같습니다.
```

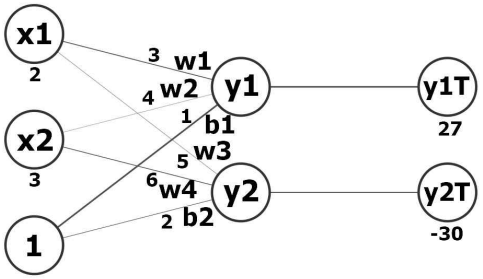
11 : 입력 층 노드의 개수를 2로 변경합니다.

2.  예제를 실행합니다. 다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=2, out_features=1, bias=True)
)
999 3.637978807091713e-12
W= tensor([[4.1429, 5.7143]])
B= tensor([1.5714])
tensor([[27.0000]])
```

04 2입력 2출력 인공 신경망 구현하기

다음은 [입력2 출력2]의 인공 신경망 학습에 사용할 신경망을 나타냅니다.



$$X = [2 \quad 3]$$
$$T = [27 \quad -30]$$
$$W = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix}$$
$$B = [1 \quad 2]$$

X를 입력 값으로, T를 목표 값으로 하여 가중치 W와 편향 B에 대해 PyTorch로 학습해 봅니다.

1. 다음과 같이 예제를 작성합니다.

344.py

```
01~04 # 이전 예제와 같습니다.
05 X = torch.FloatTensor([[2,3]]) # 입력데이터
06 YT = torch.FloatTensor([[27,-30]]) # 목표데이터(라벨)
07 W = torch.FloatTensor([[3,4],[5,6]]) # 가중치
08 B = torch.FloatTensor([1,2]) # 편향
09
10 model = nn.Sequential(
11       nn.Linear(2,2) # 입력 층, 출력 층 노드의 개수를 각각 1로 설정합니다.
12 ) # 신경망 모양 결정(W, B 내부적 준비)
13 print(model) # 신경망 모양 출력
14~끝 # 이전 예제와 같습니다.
```

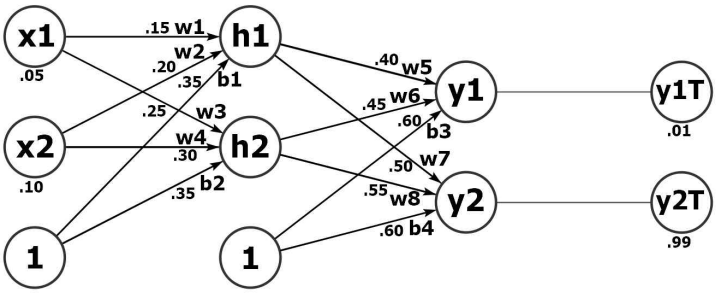
11 : 출력 층 노드의 개수를 2로 변경합니다.

2.  예제를 실행합니다. 다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=2, out_features=2, bias=True)
)
999 2.3646862246096134e-11
W= tensor([[ 4.1429,  5.7143],
        [-3.5714, -6.8571]])
B= tensor([ 1.5714, -2.2857])
tensor([[ 27.0000, -30.0000]])
```

05 2입력 2은닉 2출력 인공 신경망 구현하기

다음은 [입력2 은닉2 출력2]의 인공 신경망을 나타냅니다.



가중치와 편향 값은 그림에 있는 값을 사용합니다.

1. 다음과 같이 예제를 작성합니다.

345.py

```
01 import torch
02 import torch.nn as nn
03 import torch.optim as optim
04
05 X = torch.FloatTensor([[.05,.10]]) # 입력데이터
06 YT = torch.FloatTensor([[.01,.99]]) # 목표데이터(라벨)
07 W = torch.FloatTensor([[.15,.20],[.25,.30]]) # 가중치1
08 B = torch.FloatTensor([.35,.35]) # 편향1
09 W2 = torch.FloatTensor([[.40,.45],[.50,.55]]) # 가중치2
10 B2 = torch.FloatTensor([.60,.60]) # 편향2
11
12 model=nn.Sequential(
13         nn.Linear(2,2),
14         nn.Linear(2,2)
15 )
16 print(model)
17
18 with torch.no_grad():
19         model[0].weight = nn.Parameter(W)
20         model[0].bias = nn.Parameter(B)
21         model[1].weight = nn.Parameter(W2)
22         model[1].bias = nn.Parameter(B2)
23
24 loss_fn = nn.MSELoss() # 2공식, 오차 계산 함수
25 optimizer = optim.SGD(model.parameters(), lr=0.01)
```

```
26
27 for epoch in range(1000):
28
29       Y = model(X) # 1공식, 순전파
30       E = loss_fn(Y, YT) # 2공식, 오차계산
31       optimizer.zero_grad()
32       E.backward() # 6공식, 오차역전파
33       optimizer.step() # 7공식, 학습
34
35       if epoch%100 == 99:
36                print(epoch, E.item())
37
38 print(f'W = {model[0].weight.data}')
39 print(f'B = {model[0].bias.data}')
40 print(f'W2 = {model[1].weight.data}')
41 print(f'B2 = {model[1].bias.data}')
42
43 Y = model(X)
44 print(Y.data)
```

13 : 입력 층, 은닉 층 노드의 개수를 각각 2, 2로 설정합니다.
14 : 은닉 층, 출력 층 노드의 개수를 각각 2, 2로 설정합니다.
19~20 : 인공 신경망 0 층에 임의로 설정된 가중치와 편향을 W, B로 변경합니다.
21~22 : 인공 신경망 1 층에 임의로 설정된 가중치와 편향을 W2, B2로 변경합니다.
38~41: print 함수를 호출하여 학습이 수행된 W, B, W2, B2 행렬 값을 출력합니다.

2.  예제를 실행합니다. 다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=2, out_features=2, bias=True)
  (1): Linear(in_features=2, out_features=2, bias=True)
)
```

```
999 5.963432525568635e-11
W= tensor([[0.1432, 0.1863],
        [0.2418, 0.2836]])
B= tensor([0.2132, 0.1860])
W2= tensor([[0.2026, 0.2526],
        [0.5335, 0.5828]])
B2= tensor([-0.0956,  0.7305])
tensor([[0.0100, 0.9900]])
```

05 활성화 함수 추가하기

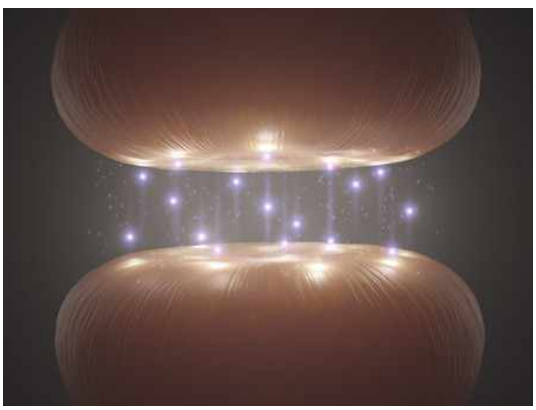
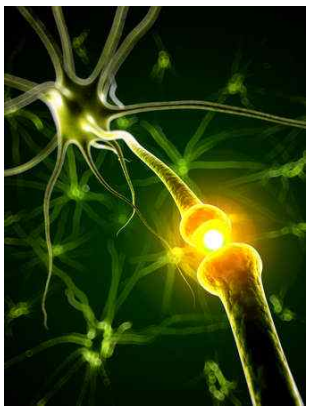
일반적으로 인공 신경의 출력 단에는 활성화 함수가 추가됩니다. 활성화 함수를 추가하면 입력된 데이터에 대해 복잡한 패턴의 학습이 가능해집니다. 또 인공 신경의 출력 값을 어떤 범위로 제한할 수도 있습니다. 여기서는 주로 사용되는 몇 가지 활성화 함수를 살펴보고, 활성화 함수가 필요한 이유에 대해서도 살펴봅니다. 그리고 활성화 함수를 추가한 인공 신경망을 구현해 봅니다.

01 활성화 함수의 필요성

여기서는 활성화 함수가 무엇인지, 활성화 함수는 왜 필요한지, 어떤 활성화 함수가 있는지 살펴봅니다.

활성화 함수는 무엇인가요?

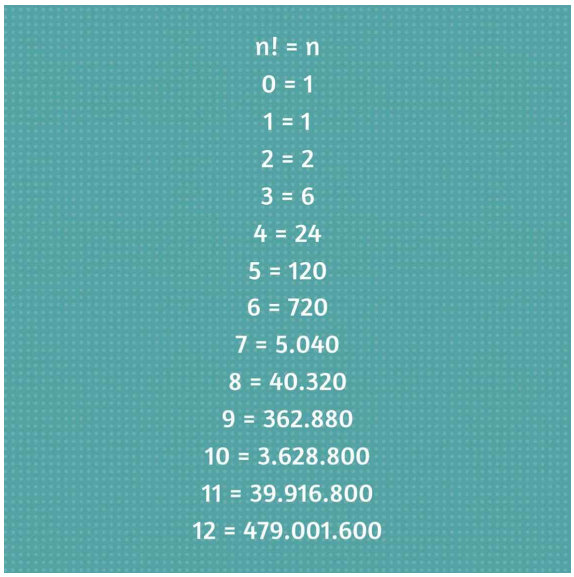
활성화 함수는 인공 신경망에 더해져 복잡한 패턴을 학습하게 해줍니다. 즉, 다양한 형태의 입력값에 대해 신경망을 거쳐 나온 출력값을 우리가 원하는 목표값에 가깝게 해 주기가 더 쉬워집니다. 우리 두뇌에 있는 생체 신경과 비교할 때, 활성화 함수는 신경 말단에서 다음 신경으로 전달될 신호를 결정하는 시냅스와 같은 역할을 합니다. 시냅스는 이전 신경 세포가 내보내는 출력 신호를 받아 다음 신경 세포가 받아들일 수 있는 입력 신호로 형태를 변경합니다. 마찬가지로 활성화 함수는 이전 인공 신경이 내보내는 출력 신호를 받아 다음 인공 신경이 받아들일 수 있는 입력 신호로 형태를 변경해 주는 역할을 합니다.

[시냅스]

활성화 함수는 왜 필요한가요?

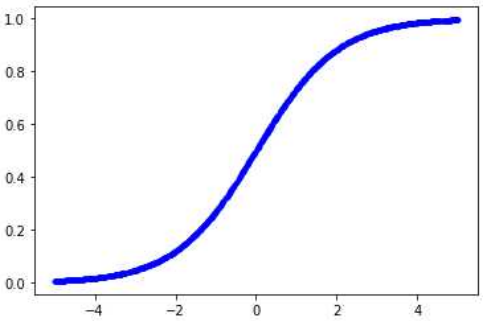
앞에서 언급했던 생물학적 유사성과는 별도로 인공 신경의 출력값을 우리가 원하는 어떤 범위로 제한해 줍니다. 이것은 활성화 함수로의 입력이 (x*w+b)이기 때문입니다. 여기서 x는 입력, w는 인공 신경의 가중치, b는 그것에 더해지는 편향입니다. (x*w+b) 값은 어떤 범위로 제한되지 않으면 신경망을 거치며 순식간에 아주 커지게 됩니다. 특히 수백만 개의 매개변수(가중치와 편향)로 구성된 아주 깊은 신경망의 경우에는 더욱 그렇습니다. 인공 신경을 거치며 반복적으로 계산되는 (x*w+b)는 factorial 연산과 같은 효과를 내며 이것은 순식간에 컴퓨터의 실수 계산 범위를 넘어서게 됩니다. 인공 신경망을 학습시키다보면 Nan이라고 표시되는 경우가 있는데 이 경우가 그런 경우에 해당합니다. 다음은 factorial 연산의 예를 보여줍니다.



어떤 활성화 함수가 있나요?

❶ 시그모이드

다음은 sigmoid 함수에 대한 그래프와 수식입니다.

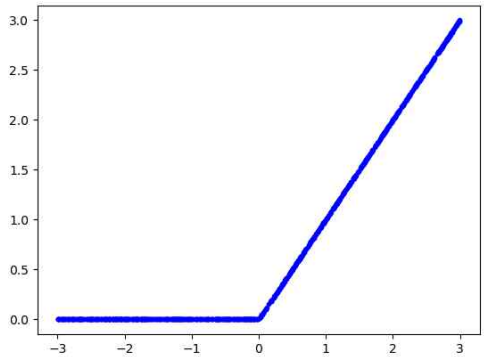


$$y = \frac{1}{1+e^{-x}} \ (-5 \leq x \leq 5)$$

시그모이드 활성화 함수는 단지 역사적인 이유로 여기에 소개되며 일반적으로 많이 사용되지 않습니다. 시그모이드 함수는 3층 정도로 구성된 인공 신경망에 적용될 때는 신경망 학습이 잘 되지만 깊은 신경망에 적용될 때는 신경망 학습이 잘 되지 않습니다. 시그모이드 함수는 계산에 시간이 걸리고, 입력값이 아무리 크더라도 출력값의 범위가 0에서 1사이로 매우 작아 신경망을 거칠수록 출력값은 점점더 작아져 0에 수렴하게 됩니다. 이것은 신경을 거치면서 신호가 점점 작아져 출력에 도달하는 신호가 아주 작거나 없어지는 것과 같습니다. 출력에 미치는 신호가 아주 작거나 없다는 것은 역으로 전달될 신호도 아주 작거나 없다는 것을 의미합니다. 시그모이드 함수는 일반적으로 0이나 1로 분류하는 이진 분류 문제에 사용됩니다. 심층 신경망에서 시그모이드 함수를 사용해야 할 경우엔 출력층에서만 사용하도록 합니다.

❷ ReLU

다음은 relu 함수에 대한 그래프와 수식입니다.

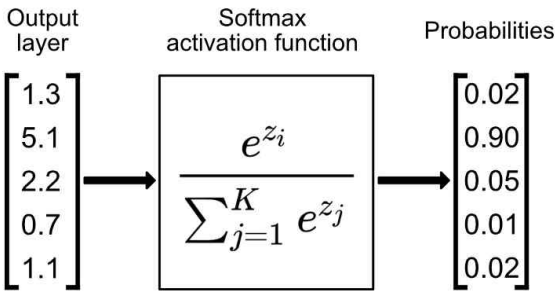


$$y = \begin{cases} x\,(x > 0) \\ 0\,(x \leq 0) \end{cases} (-3 \leq x \leq 3)$$

ReLU 함수는 딥러닝에서 가장 인기 있는 활성화 함수입니다. 특히 합성곱 신경망(CNN)에서 많이 사용됩니다. ReLU 함수는 계산이 빠르고 심층 신경망에서도 신호 전달이 잘 됩니다. ReLU 함수의 경우 입력값이 음수가 될 경우 출력이 0이 되기 때문에 이런 경우에는 어떤 노드를 완전히 죽게 하여 어떤 것도 학습하지 않게 합니다. 이러한 노드가 많으면 많을수록 신경망 전체적으로 학습이 되지 않는 단점이 있습니다. ReLU의 다른 문제는 활성화 값의 극대화입니다. 왜냐하면 ReLU의 상한값은 무한이기 때문입니다. 이것은 가끔 사용할 수 없는 노드를 만들어 학습을 방해하게 됩니다. 이러한 문제들은 초기 가중치 값을 0에 아주 가까운 값으로 고르게 할당하여 해결할 수 있습니다. 일반적으로 은닉층에는 ReLU 함수를 적용하고, 출력층은 시그모이드 함수나 소프트맥스 함수를 적용합니다.

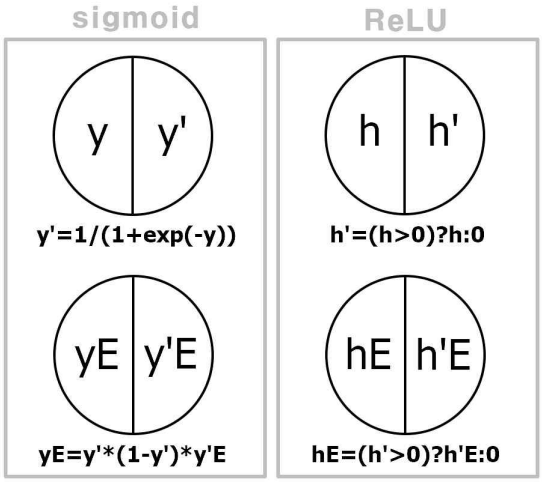
❸ 소프트맥스

소프트맥스 활성화 함수는 시그모이드 활성화 함수가 더욱 일반화된 형태입니다. 이것은 다중 클래스 분류(classification) 문제에 사용됩니다. 시그모이드 함수와 비슷하게 이것은 0에서 1사이의 값들을 생성합니다. 소프트맥스 함수는 은닉층에서는 사용하지 않으며, 다중 분류(classification) 모델에서 출력층에서만 사용합니다. 소프트맥스 함수는 뒤에서 자세히 살펴봅니다.



02 활성화 함수의 순전파와 역전파

다음 그림은 sigmoid, ReLU 활성화 함수의 순전파와 역전파 수식을 나타냅니다.



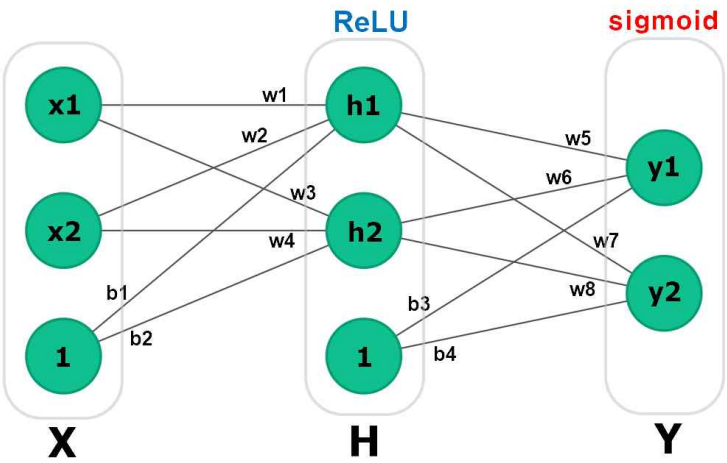
*** 역전파 수식 계산 시 순전파의 x'값도 사용됩니다. 역전파 수식은 편미분을 이용하며 유도하며 유도 과정은 생략합니다.

• sigmoid 함수의 경우 순전파 출력 x'값이 0이나 1에 가까울수록 역전파 xE값은 0에 가까워집니다. 순전파 출력 x'값이 0이나 1에 가깝다는 것은 순전파 입력값 x의 크기가 양 또는 음의 방향으로 아주 크다는 의미입니다. 입력값 x에 비해 출력값 x'가 아주 작다는 것은 입력값 x가 출력값 x'에 주는 영향이 아주 작다는 것을 의미합니다. 따라서 x'E값이 역전파를 통해 xE값에 전달되는 값도 아주 작아야 합니다.

• ReLU 함수의 경우 순전파 입력값 x값이 0보다 크면 x값이 x'로 전달되며, 0보다 작거나 같으면 0값이 x'로 전달됩니다. 역전파의 경우 순전파 출력값 x'가 0보다 크면 x'E값이 xE로 전달되며, 출력값 x'가 0보다 작거나 같으면 xE로 0이 전달됩니다. 이 경우 xE에서 전 단계의 모든 노드로 전달되는 역전파 값은 0이 됩니다. 입력값 x가 0보다 클 경우엔 출력값 x'는 x가 됩니다. 이 경우 x는 1만큼 x'로 전달된 것과 같습니다. 따라서 x'E 값은 역전파를 통해 1만큼 xE로 전달됩니다. 즉, xE = x'E가 됩니다. 입력값 x가 0보다 작거나 같은 경우엔 출력값 x'는 0이 됩니다. 이 경우 x는 0만큼 x'로 전달된 것과 같습니다. 따라서 x'E 값은 역전파를 통해 0만큼 xE로 전달됩니다. 즉, xE = 0*x'E = 0이 됩니다.

03 활성화 함수 적용하기

지금까지 정리한 sigmoid, ReLU 활성화 함수를 인공 신경망에 적용해봅니다. 먼저 딥러닝 7 공식에 적용해 보고, 다음은 PyTorch에 적용해 봅니다.



딥러닝 7 공식에 적용하기

먼저 딥러닝 7 공식에 sigmoid, ReLU 활성화 함수를 적용해 봅니다.

1. 다음과 같이 예제를 작성합니다.
353.ino

```
double x1 = 0.05, x2 = 0.10;

double w1 = 0.15, w2 = 0.20;
double w3 = 0.25, w4 = 0.30;
double b1 = 0.35, b2 = 0.35;

double w5 = 0.40, w6 = 0.45;
double w7 = 0.50, w8 = 0.55;
double b3 = 0.60, b4 = 0.60;

double y1T = 0.01, y2T = 0.99;

double lr  = 0.01;

void setup() {

        Serial.begin(115200);
```

```
for(int epoch=0;epoch<1000;epoch++) {

        double h1 = x1*w1 + x2*w2 + 1*b1;
        double h2 = x1*w3 + x2*w4 + 1*b2;
        // RELU feed forward
        h1=(h1>0)?h1:0;
        h2=(h2>0)?h2:0;

        double y1 = h1*w5 + h2*w6 + 1*b3;
        double y2 = h1*w7 + h2*w8 + 1*b4;
        // sigmoid feed forward
        y1=1/(1+exp(-y1));
        y2=1/(1+exp(-y2));

        double E = ((y1-y1T)*(y1-y1T) + (y2-y2T)*(y2-y2T))/2;

        double y1E = y1 - y1T;
        double y2E = y2 - y2T;
        // sigmoid back propagation
        y1E=y1*(1-y1)*y1E;
        y2E=y2*(1-y2)*y2E;

        double w5E = y1E*h1;
        double w6E = y1E*h2;
        double w7E = y2E*h1;
        double w8E = y2E*h2;
        double b3E = y1E*1;
        double b4E = y2E*1;

        double h1E = y1E*w5 + y2E*w7;
        double h2E = y1E*w6 + y2E*w8;
        // RELU back propagation
        h1E=(h1>0)?h1E:0;
        h2E=(h2>0)?h2E:0;

        double w1E = h1E*x1;
        double w2E = h1E*x2;
        double w3E = h2E*x1;
        double w4E = h2E*x2;
        double b1E = h1E*1;
```

```
                double b2E = h2E*1;

                w5 -= lr*w5E;
                w6 -= lr*w6E;
                w7 -= lr*w7E;
                w8 -= lr*w8E;
                b3 -= lr*b3E;
                b4 -= lr*b4E;

                w1 -= lr*w1E;
                w2 -= lr*w2E;
                w3 -= lr*w3E;
                w4 -= lr*w4E;
                b1 -= lr*b1E;
                b2 -= lr*b2E;

                Serial.print("epoch = "); Serial.println(epoch);
                Serial.print("y1 : "); Serial.println(y1, 3);
                Serial.print("y2 : "); Serial.println(y2, 3);

                if(E<0.0000001) break;

        }

}

void loop() {
}
```

2 : math 라이브러리에서 exp 파일을 불러옵니다.
26~27 : RELU 순전파 동작을 추가합니다.
32~33 : sigmoid 순전파 동작을 추가합니다.
40~41 : sigmoid 역전파 동작을 추가합니다.
53~54 : RELU 역전파 동작을 추가합니다.
77~80 : 매 100회마다 회기, y1, y2 값을 출력합니다.

2. 

업로드를 수행합니다.

3. [시리얼 모니터] 버튼을 클릭하고, 통신 속도를 115200으로 맞춰줍니다.



4. 출력결과를 확인합니다.

```
epoch = 999
 y1 : 0.359368
 y2 : 0.809741
w1,w3 = 0.149033,0.247115
w2,w4 = 0.198065,0.294230
b1,b2 = 0.330653,0.292297
w5,w7 = 0.009232,0.629159
w6,w8 = 0.063927,0.677595
b3,b4 = -0.603755,0.997324
```

학습 회수에 따라 o1, o2값이 바뀌는 것을 확인합니다. o1, o2값이 각각 0.01, 0.99에 가까워지는 것을 확인합니다. 입력값 0.05, 0.10에 대해 목표값은 0.01, 0.99입니다.

5. epoch 값의 범위를 다음과 같이 수정한 후, 업로드를 수행합니다.

353_2.ino

```
        for(int epoch=0;epoch<100000;epoch++) {
```

6. 출력결과를 확인합니다.

```
epoch = 99999
 y1 : 0.013750
 y2 : 0.987571
w1,w3 = 0.198118,0.291117
w2,w4 = 0.296234,0.382345
b1,b2 = 1.312289,1.172902
w5,w7 = -0.956824,1.076556
w6,w8 = -0.813884,1.083658
b3,b4 = -1.981831,1.591643
```

PyTorch에 적용하기

다음은 PyTorch 예제에 적용해 봅니다.

1. 345.py 파일을 복사하여 다음과 같이 예제를 수정합니다.
353_2.py

```
01 import torch
02 import torch.nn as nn
03 import torch.optim as optim
04
05 X = torch.FloatTensor([[.05,.10]]) # 입력데이터
06 YT = torch.FloatTensor([[.01,.99]]) # 목표데이터(라벨)
07 W = torch.FloatTensor([[.15,.20],[.25,.30]]) # 가중치1
08 B = torch.FloatTensor([.35,.35]) # 편향1
09 W2 = torch.FloatTensor([[.40,.45],[.50,.55]]) # 가중치2
10 B2 = torch.FloatTensor([.60,.60]) # 편향2
11
12 model=nn.Sequential(
13         nn.Linear(2,2),
14         nn.ReLU(),
15         nn.Linear(2,2),
16         nn.Sigmoid()
17 )
18 print(model)
19
20 with torch.no_grad():
21         model[0].weight = nn.Parameter(W)
22         model[0].bias = nn.Parameter(B)
23         model[2].weight = nn.Parameter(W2)
24         model[2].bias = nn.Parameter(B2)
25
26 loss_fn = nn.MSELoss() # 2공식, 오차 계산 함수
27 optimizer = optim.SGD(model.parameters(), lr=0.01)
28
29 for epoch in range(1000):
30
31         Y = model(X) # 1공식, 순전파
32         E = loss_fn(Y, YT) # 2공식, 오차계산
33         optimizer.zero_grad()
34         E.backward() # 6공식, 오차역전파
```

```
35        optimizer.step() # 7공식, 학습
36
37        if epoch%100 == 99:
38                print(epoch, E.item())
39
40 print(f'W = {model[0].weight.data}')
41 print(f'B = {model[0].bias.data}')
42 print(f'W2 = {model[2].weight.data}')
43 print(f'B2 = {model[2].bias.data}')
44
45 Y = model(X)
46 print(Y.data)
```

14 : 은닉 층에 활성화 함수 ReLU를 적용합니다.

16 : 출력 층에 활성화 함수 Sigmoid를 적용합니다.

2.  예제를 실행합니다. 다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=2, out_features=2, bias=True)
  (1): ReLU()
  (2): Linear(in_features=2, out_features=2, bias=True)
  (3): Sigmoid()
)
```

```
999 0.0772756040096283
W= tensor([[0.1490, 0.1981],
        [0.2471, 0.2942]])
B= tensor([0.3307, 0.2923])
W2= tensor([[0.0092, 0.0639],
        [0.6292, 0.6776]])
B2= tensor([-0.6038,  0.9973])
tensor([[0.3591, 0.8098]])
```

3. 다음과 같이 예제를 수정한 후, 실행해 봅니다.

353_3.py

```
29 for epoch in range(100000):
```

다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=2, out_features=2, bias=True)
  (1): ReLU()
  (2): Linear(in_features=2, out_features=2, bias=True)
  (3): Sigmoid()
)
```

```
99999 9.983300515159499e-06
W= tensor([[0.1981, 0.2962],
        [0.2911, 0.3823]])
B= tensor([1.3123, 1.1729])
W2= tensor([[-0.9568, -0.8139],
        [ 1.0766,  1.0837]])
B2= tensor([-1.9818,  1.5916])
tensor([[0.0138, 0.9876]])
```

04 출력층에 linear 함수 적용해 보기

여기서는 출력층 함수에 linear 함수를 적용해 봅니다. linear 함수는 출력단의 값을 그대로 내보내는 함수입니다. linear 함수를 적용하여 학습을 수행할 경우 선형 회귀라고 합니다.

딥러닝 7 공식에 적용하기

먼저 딥러닝 7 공식에 linear 활성화 함수를 적용해 봅니다.

1. 다음과 같이 예제를 작성합니다. 353.ino 파일을 복사한 후, 수정합니다.
354.ino

```
29              // sigmoid feed forward
30              // y1=1/(1+exp(-y1));
31              // y2=1/(1+exp(-y2));
32
33              double E = ((y1-y1T)*(y1-y1T) + (y2-y2T)*(y2-y2T))/2;
34
35              double y1E = y1 - y1T;
36              double y2E = y2 - y2T;
37              // sigmoid back propagation
38              // y1E=y1*(1-y1)*y1E;
39              // y2E=y2*(1-y2)*y2E;
```

30~31 : 주석 처리합니다.
38~39 : 주석 처리합니다.

2.  업로드를 수행합니다.

3. [시리얼 모니터] 버튼을 클릭하고, 통신 속도를 115200으로 맞춰줍니다.



4. 출력결과를 확인합니다.

```
epoch = 599
 y1 : 0.010858
 y2 : 0.989648
w1,w3 = 0.143157,0.241800
w2,w4 = 0.186315,0.283601
b1,b2 = 0.213147,0.186007
w5,w7 = 0.202730,0.533461
w6,w8 = 0.252676,0.582771
b3,b4 = -0.095253,0.730397
```

PyTorch에 적용하기

다음은 PyTorch 예제에 적용해 봅니다.

1. 353_2.py 파일을 복사하여 다음과 같이 예제를 수정합니다.
354_2.py

```
01~11 # 이전 예제와 같습니다.
12 model=nn.Sequential(
13         nn.Linear(2,2),
14         nn.ReLU(),
15         nn.Linear(2,2),
16         #nn.Sigmoid()
17 )
18 print(model)
18~끝 # 이전 예제와 같습니다.
```

16 : 출력 층에 활성화 함수 사용하지 않습니다. 이렇게 하면 15줄의 결과가 그대로 출력이 되며, linear하다고 합니다.

2. 계속해서 다음 부분을 수정합니다.

```
33 for epoch in range(600):
```

33 : 학습 횟수는 600으로 합니다.

2.  예제를 실행합니다. 다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=2, out_features=2, bias=True)
  (1): ReLU()
  (2): Linear(in_features=2, out_features=2, bias=True)
)
```

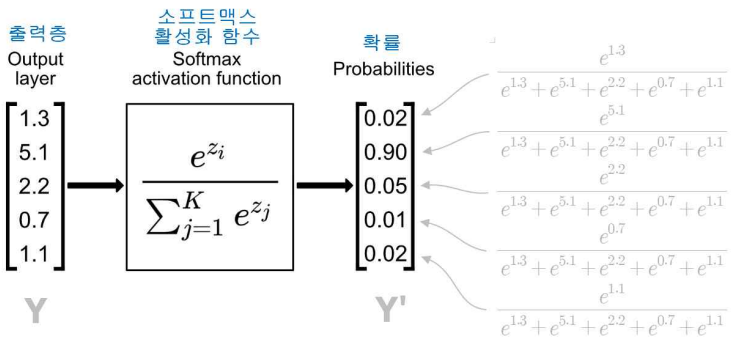
```
599 4.304388596665376e-07
W= tensor([[0.1432, 0.1863],
        [0.2418, 0.2836]])
B= tensor([0.2131, 0.1860])
W2= tensor([[0.2028, 0.2528],
        [0.5334, 0.5827]])
B2= tensor([-0.0949,  0.7302])
tensor([[0.0108, 0.9897]])
```

05 softmax 활성화 함수/cross entropy 오차 함수 살펴보기

이전 단원에서 우리는 은닉 신경과 출력 신경에 relu, sigmoid 활성화 함수를 적용해 보았습니다. 이 단원에서는 출력 신경의 활성화 함수를 소프트맥스(softmax)로 변경해 봅니다. softmax 활성화 함수는 크로스 엔트로피 오차(cross entropy error) 함수와 같이 사용되며, 분류(classification)에 사용됩니다.

softmax와 cross entropy

다음은 출력층에서 활성화 함수로 사용되는 소프트맥스(softmax) 함수를 나타냅니다.



소프트맥스 함수는 출력층에서 사용되는 활성화함수로 다중 분류(classification)를 위해 주로 사용됩니다. 소프트맥스 함수는 확률의 총합이 1이 되도록 만든 함수이며 아래에 나타낸 크로스 엔트로피 오차 함수와 같이 사용됩니다. 크로스 엔트로피 오차 함수의 경우 목표값(ykT) 중 하나만 1, 나머지는 0 값을 갖도록 목표값을 설정해야 합니다.

$$E = -\sum_{k}(y_{kT} * \log(y_k))$$

우리는 앞에서 다음과 같은 평균 제곱 오차 함수를 살펴보았습니다.

$$E = \sum_{k}\frac{1}{2}(y_k - y_{kT})^2$$

평균 제곱 오차 함수의 경우 역전파 시 전파되는 오차가 다음과 같이 예측값과 목표값의 차인 것을 우리는 이미 앞에서 살펴보았습니다.

$$y_{kE} = y_k - y_{kT}$$

소프트맥스 함수는 크로스 엔트로피 함수와 같이 사용될 때만 역전파 시 소프트맥스 함수를 역으로 거쳐 전파되는 오차가 다음과 같이 예측값과 목표값의 차가 됩니다.

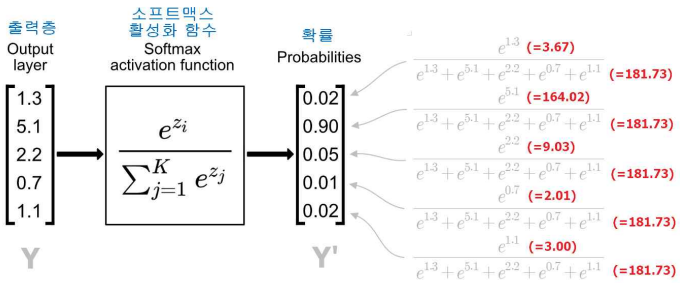
$$y_{kE} = y_k - y_{kT}$$

*** 이 수식은 크로스 엔트로피 함수와 소프트맥스 함수에 대해 차례대로 편미분을 적용하여 얻어진 수식이며 이 책에서는 수식에 대한 유도 과정은 생략합니다.

그래서 일반적으로 소프트맥스 함수를 활성화 함수로 사용할 경우 오차 함수는 크로스 엔트로피 오차 함수가 됩니다.

softmax 함수 구현해 보기

여기서는 다음 그림에 대해 softmax 함수를 구현해 봅니다.



출력층에서 Y의 각 항목은 소프트맥스 활성화 함수를 거쳐 Y'의 각 항목으로 변환됩니다. Y'의 모든 항목의 합은 1이 됩니다.

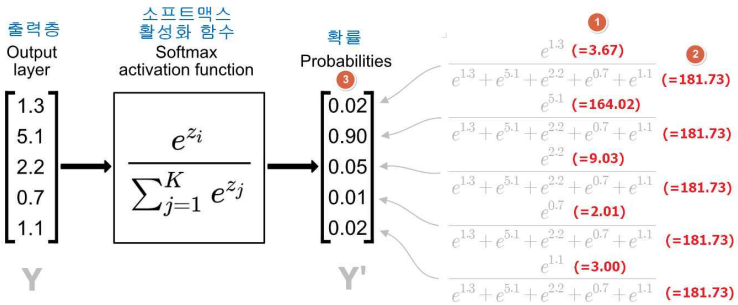
다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
355.ino

```
01 #include <math.h>
02
03 void setup() {
04
05         Serial.begin(115200);
06
07         Serial.println(exp(1.3));
08         Serial.println(exp(5.1));
09         Serial.println(exp(2.2));
10         Serial.println(exp(0.7));
11         Serial.println(exp(1.1));
12
13         double sumY = exp(1.3)+exp(5.1)+exp(2.2)+exp(0.7)+exp(1.1);
14         Serial.println(sumY);
15
16         Serial.println(exp(1.3)/sumY);
17         Serial.println(exp(5.1)/sumY);
18         Serial.println(exp(2.2)/sumY);
19         Serial.println(exp(0.7)/sumY);
20         Serial.println(exp(1.1)/sumY);
21
22 }
23
24 void loop() {
25 }
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
[[  3.67, 164.02,   9.03,   2.01,   3.00]] ❶
181.73 ❷
[[0.02, 0.90, 0.05, 0.01, 0.02]] ❸
```

다음 그림의 ❶, ❷, ❸과 출력 결과의 ❶, ❷, ❸을 비교해 봅니다.



❶ 첫 번째 결과는 Y의 각 항목에 대해 exp 함수를 적용한 결과입니다.
❷ 두 번째 결과는 첫 번째 결과의 모든 항목을 더한 값입니다.
❸ 세 번째 결과는 첫 번째 결과의 값들을 두 번째 결과의 값으로 나눈 결과입니다.

softmax 함수의 분모 크기 줄이기

앞에서 우리는 softmax 함수의 분모를 다음과 같이 계산하여 181.73 값을 얻었습니다.

$$e^{1.3}+e^{5.1}+e^{2.2}+e^{0.7}+e^{1.1}=181.73$$

이 수식에서 Y의 항목 값이 하나라도 지나치게 클 경우의 계산 결과는 아주 큰 숫자가 되어 계산할 수 있는 실수의 범위를 벗어날 수 있습니다. 이런 경우 그 위험을 줄이기 위해 Y 행렬의 각 항목을 다음과 같이 바꿀 수 있습니다.

$$\begin{bmatrix} 1.3-5.1 \\ 5.1-5.1 \\ 2.2-5.1 \\ 0.7-5.1 \\ 1.1-5.1 \end{bmatrix} = \begin{bmatrix} -3.8 \\ 0.0 \\ -2.9 \\ -4.4 \\ -4.0 \end{bmatrix}$$

Y -5.1 ⟶ Yc

위 그림에서 왼쪽의 Y 행렬에서 가장 큰 항목(5.1)을 찾아 행렬의 각 항목에 대해 빼주어 오른쪽의 Yc 행렬로 변환한 후, softmax 함수를 적용하여도 결과는 같게 됩니다. 다음은 이 과정에 대한 구체적인 예를 보여줍니다.

$$\frac{e^{1.3}}{e^{1.3}+e^{5.1}+e^{2.2}+e^{0.7}+e^{1.1}} \quad ❶$$

❷

$$=\frac{e^{1.3}/e^{5.1}}{(e^{1.3}+e^{5.1}+e^{2.2}+e^{0.7}+e^{1.1})/e^{5.1}} \quad ❸ \; ❹$$

$$=\frac{e^{1.3}/e^{5.1}}{e^{1.3}/e^{5.1}+e^{5.1}/e^{5.1}+e^{2.2}/e^{5.1}+e^{0.7}/e^{5.1}+e^{1.1}/e^{5.1}} \quad ❺ \; ❻$$

$$=\frac{e^{(1.3-5.1)}}{e^{(1.3-5.1)}+e^{(5.1-5.1)}+e^{(2.2-5.1)}+e^{(0.7-5.1)}+e^{(1.1-5.1)}} \quad ❼ \; ❽$$

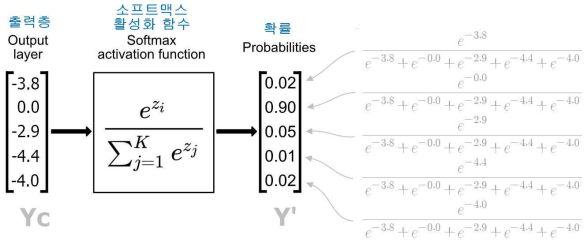
$$=\frac{e^{-3.8}}{e^{-3.8}+e^{0.0}+e^{-2.9}+e^{-4.4}+e^{-4.0}} \quad ❾$$

위에서 ❶ 수식은 최초의 Y 행렬의 첫 번째 항목에 대해 softmax 함수를 적용한 수식입니다. 여기서 ❷ 가장 큰 항목을 찾아 ❸, ❹와 같이 분자와 분모를 각각 나누어 줍니다. 그러면 ❺, ❻과 같이 분자와 분모를 바꿀 수 있습니다. ❺, ❻은 ❼, ❽과 같이 계산할 수 있으며, 결과적으로 ❾와 같이 됩니다. 결과적으로 ❶에서 최초의 Y의 첫 번째 항목 1.3에 대해 softmax 함수를 적용한 결과는 ❾에서 변환된 Yc의 첫 번째 항목 -3.8에 softmax 함수를 적용한 결과와 같게 됩니다. ❾에서 분모의 합은 다음과 같이 계산하여 1.11이 됩니다.

$$e^{-3.8}+e^{-0.0}+e^{-2.9}+e^{-4.4}+e^{-4.0}=1.11$$

이렇게 하면 softmax 함수의 분모가 지나치게 커지는 것을 막을 수 있습니다.

다음 그림은 변환된 Yc 행렬에 대해 softmax 함수를 적용하는 과정을 나타내는 그림입니다.



다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.

355_2.ino

```
#include <math.h>

void setup() {

        Serial.begin(115200);

        Serial.println(exp(1.3-5.1));
        Serial.println(exp(5.1-5.1));
        Serial.println(exp(2.2-5.1));
        Serial.println(exp(0.7-5.1));
        Serial.println(exp(1.1-5.1));

        double sumY = exp(1.3-5.1)+exp(5.1-5.1)+exp(2.2-5.1)+exp(0.7-5.1)+exp(1.1-5.1);
        Serial.println(sumY);

        Serial.println(exp(1.3-5.1)/sumY);
        Serial.println(exp(5.1-5.1)/sumY);
        Serial.println(exp(2.2-5.1)/sumY);
        Serial.println(exp(0.7-5.1)/sumY);
        Serial.println(exp(1.1-5.1)/sumY);

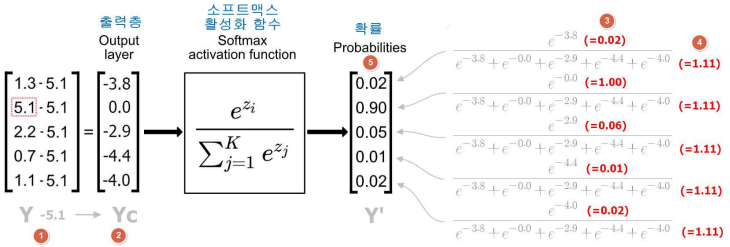
}

void loop() {
}
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

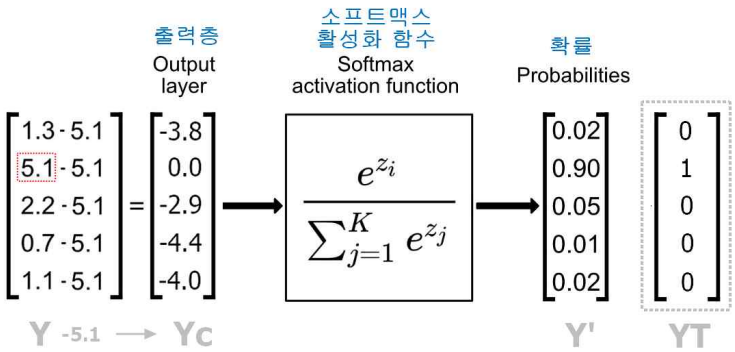
```
5.10 ❶
[[-3.80,  0.00, -2.90, -4.40, -4.00]]❷
[[0.02, 1.00, 0.06, 0.01, 0.02]] ❸
1.11 ❹
[[0.02, 0.90, 0.05, 0.01, 0.02]] ❺
```

다음 그림의 ❶, ❷, ❸, ❹, ❺와 출력 결과의 ❶, ❷, ❸, ❹, ❺를 비교해 봅니다.



cross entropy 오차 구현해 보기

softmax 함수를 사용할 경우 다음과 같이 하나의 목표값만 1이고 나머지는 0이 됩니다.



softmax 함수는 분류(classification)를 위해 사용하며 결과값 중 하나의 항목만 1에 가깝고 나머지는 0에 가까워지도록 학습하게 됩니다. 소프트맥스 함수는 확률의 총합이 1이 되도록 만든 함수이며 아래에 나타낸 크로스 엔트로피 오차 함수와 같이 사용합니다.

$$E = -\sum_{k}(y_{kT} * \log(y_k))$$

위 그림의 경우 오차는 다음과 같이 계산됩니다.

$$\begin{aligned} E &= -(0*\log(0.02)+1*\log(0.90)+0*\log(0.05)+0*\log(0.01)+0*\log(0.02)) \\ &= -(1*\log(0.90)) = 0.11 \end{aligned}$$

다음과 같이 아두이노 스케치를 작성하여 테스트해 봅니다.
355_3.ino

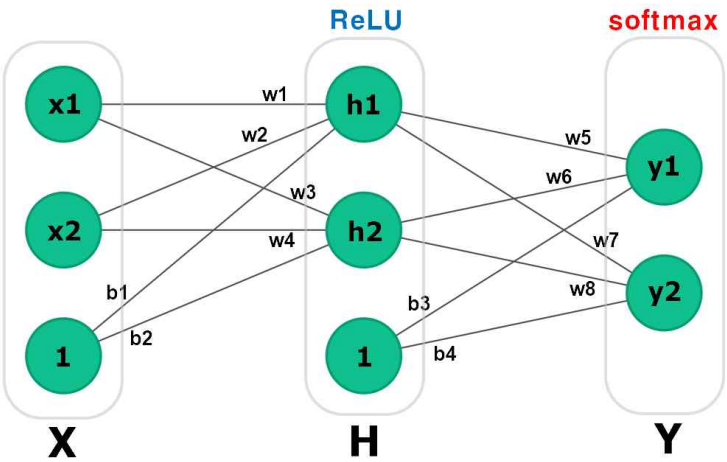
```
01 #include <math.h>
02
03 void setup() {
04
05   Serial.begin(115200);
06
07   Serial.println(-(1*log(0.90)));
08
09 }
10
11 void loop() {
12 }
```

예제를 보드에 업로드한 후, 시리얼 모니터로 결과를 확인해 봅니다.

```
[[-0.00, -0.11, -0.00, -0.00, -0.00]]
0.11
```

06 softmax 활성화 함수/cross entropy 오차 함수 적용하기

지금까지 정리한 softmax 활성화 함수와 cross entropy 오차 함수를 인공 신경망에 적용해봅니다. 먼저 딥러닝 7 공식에 적용해 보고, 다음은 PyTorch에 적용해 봅니다.



딥러닝 7 공식에 적용하기

먼저 딥러닝 7 공식에 softmax 활성화 함수와 cross entropy 오차 함수를 적용해 봅니다.

1. 다음과 같이 예제를 작성합니다. 353.py 파일을 복사한 후, 수정합니다.
356.ino

```
01 double x1 = 0.05, x2 = 0.10;
02
03 double w1 = 0.15, w2 = 0.20;
04 double w3 = 0.25, w4 = 0.30;
05 double b1 = 0.35, b2 = 0.35;
06
07 double w5 = 0.40, w6 = 0.45;
08 double w7 = 0.50, w8 = 0.55;
09 double b3 = 0.60, b4 = 0.60;
10
11 double y1T = 0, y2T = 1; // 크로스 엔트로피는 하나의 목표 값만 1, 나머지는 0
12
13 double lr  = 0.01;
14
15 void setup() {
16
17   Serial.begin(115200);
18
19   for(int epoch=0;epoch<200000;epoch++) {
```

```
20
21      double h1 = x1*w1 + x2*w2 + 1*b1;
22      double h2 = x1*w3 + x2*w4 + 1*b2;
23      // RELU feed forward
24      h1=(h1>0)?h1:0;
25      h2=(h2>0)?h2:0;
26
27      double y1 = h1*w5 + h2*w6 + 1*b3;
28      double y2 = h1*w7 + h2*w8 + 1*b4;
29      // softmax feed forward
30      double yMax=(y1>y2)?y1:y2;
31      y1-=yMax;
32      y2-=yMax;
33      double sumY=exp(y1)+exp(y2);
34      y1=exp(y1)/sumY;
35      y2=exp(y2)/sumY;
36
37      double E = -(y1T*log(y1)+y2T*log(y2)); // cross entropy error
38
39      double y1E = y1 - y1T;
40      double y2E = y2 - y2T;
41      // softmax back propagation
42      // do nothing
43
44      double w5E = y1E*h1;
45      double w6E = y1E*h2;
46      double w7E = y2E*h1;
47      double w8E = y2E*h2;
48      double b3E = y1E*1;
49      double b4E = y2E*1;
50
51      double h1E = y1E*w5 + y2E*w7;
52      double h2E = y1E*w6 + y2E*w8;
53      // RELU back propagation
54      h1E=(h1>0)?h1E:0;
55      h2E=(h2>0)?h2E:0;
56
57      double w1E = h1E*x1;
58      double w2E = h1E*x2;
59      double w3E = h2E*x1;
60      double w4E = h2E*x2;
61      double b1E = h1E*1;
62      double b2E = h2E*1;
63
64      w5 -= lr*w5E;
```

```
65      w6 -= lr*w6E;
66      w7 -= lr*w7E;
67      w8 -= lr*w8E;
68      b3 -= lr*b3E;
69      b4 -= lr*b4E;
70
71      w1 -= lr*w1E;
72      w2 -= lr*w2E;
73      w3 -= lr*w3E;
74      w4 -= lr*w4E;
75      b1 -= lr*b1E;
76      b2 -= lr*b2E;
77
78      Serial.print("epoch = "); Serial.println(epoch);
79      Serial.print("y1 : "); Serial.println(y1, 3);
80      Serial.print("y2 : "); Serial.println(y2, 3);
81
82      if(E<0.0000001) break;
83
84    }
85
86 }
87
88 void loop() {
89 }
```

46 : 목표값을 각각 0과 1로 변경합니다.

54 : 출력층의 활성화 함수를 소프트맥스로 설정합니다.

56 : 오차 계산 함수를 크로스 엔트로피 오차 함수로 설정합니다.

2. 

업로드를 수행합니다.

3. 출력 결과를 확인합니다.

```
epoch = 99999
 y1 : 0.000052
 y2 : 0.999948
w1,w3 = 0.209796,0.311644
w2,w4 = 0.319592,0.423376
b1,b2 = 1.546079,1.583957
w5,w7 = -0.635543,1.535484
w6,w8 = -0.621608,1.621555
b3,b4 = -0.767205,1.967242
```

PyTorch에 적용하기

다음은 PyTorch 예제에 적용해 봅니다.

1. 353_2.py 파일을 복사하여 다음과 같이 예제를 수정합니다.
356_2.py

```
01 import torch
02 import torch.nn as nn
03 import torch.optim as optim
04
05 X = torch.FloatTensor([[.05,.10]]) # 입력데이터
06 YT = torch.FloatTensor([[0,1]]) # 목표데이터(라벨)
07 W = torch.FloatTensor([[.15,.20],[.25,.30]]) # 가중치1
08 B = torch.FloatTensor([.35,.35]) # 편향1
09 W2 = torch.FloatTensor([[.40,.45],[.50,.55]]) # 가중치2
10 B2 = torch.FloatTensor([.60,.60]) # 편향2
11
12 model=nn.Sequential(
13         nn.Linear(2,2),
14         nn.ReLU(),
15         nn.Linear(2,2),
16 #         nn.Sigmoid()
17 )
18 print(model)
19
20 with torch.no_grad():
21         model[0].weight = nn.Parameter(W)
22         model[0].bias = nn.Parameter(B)
23         model[2].weight = nn.Parameter(W2)
24         model[2].bias = nn.Parameter(B2)
25
26 loss_fn = nn.CrossEntropyLoss() # 2공식, 오차 계산 함수
27 optimizer = optim.SGD(model.parameters(), lr=0.01)
28
29 for epoch in range(100000):
30
31         Y = model(X) # 1공식, 순전파
32         E = loss_fn(Y, YT) # 2공식, 오차계산
33         optimizer.zero_grad()
34         E.backward() # 6공식, 오차역전파
```

```
35        optimizer.step() # 7공식, 학습
36
37        if epoch%100 == 99:
38                print(epoch, E.item())
39
40 print(f'W = {model[0].weight.data}')
41 print(f'B = {model[0].bias.data}')
42 print(f'W2 = {model[2].weight.data}')
43 print(f'B2 = {model[2].bias.data}')
44
45 Y = model(X)
46 Y = nn.functional.softmax(Y, dim=1)
47 print(Y.data)
```

06 : 목표값을 각각 0과 1로 변경합니다.

16 : 출력층에 활성화 함수는 사용하지 않습니다. PyTorch의 경우 cross entropy 오차 함수를 사용할 경우 출력층에 softmax 함수를 따로 사용하지 않습니다. PyTorch에서 제공하는 nn.CrossEntropyLoss 함수 내에는 softmax 함수가 구현되어 있기 때문입니다.

26 : 오차 함수를 nn.CrosseEtropyLoss로 설정합니다. PyTorch CrossEntropyLoss 함수는 다음과 같이 softmax 함수를 내부에 포함하고 있습니다. 그래서 16줄에 출력층 함수를 softmax 함수를 따로 써 주지 않습니다.

$$\text{loss}(x, class) = \underbrace{-\log\left(\overbrace{\frac{\exp(x[class])}{\sum_j \exp(x[j])}}^{\text{softmax}}\right)}_{\text{CrossEntropyLoss}} = -x[class] + \log\left(\sum_j \exp(x[j])\right)$$

29 : 학습 횟수는 100000으로 합니다.

46 : nn.functional.softmax 함수를 이용하여 Y값을 softmax 출력 형태로 변경합니다.

2.  예제를 실행합니다. 다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Linear(in_features=2, out_features=2, bias=True)
  (1): ReLU()
  (2): Linear(in_features=2, out_features=2, bias=True)
)
```

```
99999 5.185469490243122e-05
W= tensor([[0.2098, 0.3196],
        [0.3116, 0.4234]])
B= tensor([1.5461, 1.5840])
W2= tensor([[-0.6355, -0.6216],
        [ 1.5355,  1.6216]])
B2= tensor([-0.7672,  1.9672])
tensor([[5.1898e-05, 9.9995e-01]])
```

출력층 활성화 함수와 오차 함수의 관계

다음은 출력층 활성화 함수에 따라 사용할 수 있는 오차 함수입니다.

Problem type	Last-layer activation	Loss function	Example
Binary classification	`sigmoid`	`binary_crossentropy`	Dog vs cat, Sentiemnt analysis(pos/neg)
Multi-class, single-label classification	`softmax`	`categorical_crossentropy`	MNIST has 10 classes single label (one prediction is one digit)
Multi-class, multi-label classification	`sigmoid`	`binary_crossentropy`	News tags classification, one blog can have multiple tags
Regression to arbitrary values	`None`	`mse`	Predict house price(an integer/float point)
Regression to values between 0 and 1	`sigmoid`	`mse` or `binary_crossentropy`	Engine health assessment where 0 is broken, 1 is new